

OPERATIONS RUNBOOK · DRAFT FOR REHEARSAL

VMware 미러 기반 Oracle Linux 9.6 전환 작업지침서

AS-IS EL7 계열 또는 Rocky Linux 8 추정 3-노드 환경의 미러 인수, 클러스터 재구성, 선택 데이터 복원, 검증 및 롤백 절차

문서 버전 1.1

작성 기준일 2026-09-03

대상: 3 VM / admin 운영

최종 승인 전 실행 금지

확정 전환 모델: 목표 OS는 Oracle Linux 9.6이며 AS-IS는 EL7 계열 또는 Rocky Linux 8로 추정되나 작업 전 실측이 필수다. MariaDB는 미러 DB를 그대로 기동하고 Scheduler가 간섭하는 데이터만 INSERT SQL로 추출해 해당 데이터를 비운 뒤 재적재한다. Solr collection은 반드시 BACKUP/RESTORE로 이관한다. Redis는 별도 데이터 이관 없이 미러본으로 클러스터를 정상화하고 시스템 배포 기능으로 cache를 재배포한다. Elasticsearch는 미러 데이터를 보관만 한 뒤 bin data 경로로 재클러스터링하고 Snapshot/Restore하며 ILM·SLM은 명시적으로 이관·검증한다.

이 런북 전체를 아주 쉽게 읽는 방법

이 문서는 '컴퓨터 세 대를 새 건물로 옮기는 작업 설명서'입니다.

원본 서버는 **source**, 새 환경은 **target**입니다. VM 미러는 컴퓨터 본체를 복사하는 일이고, MariaDB·Redis·Solr·Elasticsearch 데이터는 각각 알맞은 방법으로 확인하고 복원합니다.

검은 명령어 상자는 실제 명령의 원문이고, 이 파란 상자는 그 명령이 하는 일을 풀어쓴 설명입니다. [조회]는 읽기, [변경]은 상태를 바꾸는 행동이므로 승인 없이 실행하지 않습니다.

가장 중요한 약속: 원본과 복사본을 같은 IP로 동시에 켜지 않고, 백업과 롤백 방법을 먼저 만든 뒤, target을 격리해서 테스트합니다.

목차

1. 목적·범위
2. 작업 원칙과 전환 모델
3. 현행 기준정보
4. 역할·사전조건·변수
5. G0: 사전 채증과 B0 백업
6. M0: VMware 미리 인수
7. R0: OL9.6 대상 준비
8. MariaDB 미리 기동·Scheduler 데이터 재적재
9. Redis 재클러스터링·cache 재배포
10. ZooKeeper/Solr 재클러스터링·복원
11. Elasticsearch/Kibana 복원
12. 애플리케이션·Scheduler 기동
13. T0: 시험 항목
14. B1: 최종 데이터 동기화
15. 중지·기동 순서
16. GO/NO-GO와 롤백
17. 부록 A: Scheduler 데이터 분류
18. 부록 B: 증적·인수 체크리스트

1. 목적·범위

아주 쉽게 설명하면: 1. 목적·범위

이 부분은 이사 계획의 지도입니다.

이 문서는 현재 서버에서 새 환경으로 옮길 때 어디까지 다루고 어디부터는 다른 담당자와 상의해야 하는지를 정합니다. '포함'은 이번 작업의 범위이고, '별도 협의'는 네트워크·VMware·운영 중단처럼 다른 담당자의 승인이 필요한 부분이며, '금지'는 실수하면 원본 데이터를 잃을 수 있어 절대 함부로 하지 않는 행동입니다.

여기서 가장 중요한 사실은 목표가 Oracle Linux 9.6이라는 점과, 원본 OS 종류는 추측하지 말고 실제 서버에서 다시 확인해야 한다는 점입니다. 즉, 이 문서는 '무조건 그대로 실행하는 요리법'이 아니라, 현장 확인 결과에 따라 다음 행동을 선택하는 안전한 작업지도입니다.

이 부분에서 확인할 것: 실제 결과물: 작업 범위표, 원본 OS 확인 결과, 담당자 승인.

이 지침서는 EL7 계열 또는 Rocky Linux 8로 추정되는 운영 또는 운영 유사 3대 VM을 VMware 방식으로 복제해 Oracle Linux 9.6에서 동일 애플리케이션을 검증하고 전환하는 절차를 정의한다. AS-IS OS는 아직 확인되지 않았으므로 정확한 배포판·release·kernel·architecture를 작업 전 채증하며, 실측값이 본 문서의 추정과 다르면 호환성 판정을 다시 수행한다.

포함

파일·JAR 이관, OS/JDK/라이브러리 검증, MariaDB 미러 기동과 Scheduler 영향 데이터 재적재, Redis 재클러스터링과 cache 재배포, Solr collection 필수 이관, Elasticsearch Snapshot/Restore와 ILM·SLM 검증, 기능·장애 시험.

별도 협의

VMware 미러 생성과 OL9.6 전환 자체, L4/VIP 전환, 운영 DB 이중화·복제 솔루션, 방화벽·보안 정책 승인, 최종 업무 중단 공지.

금지

소스·대상 동시 동일 IP 사용, 미러된 클러스터 데이터의 무검증 기동, 리허설 데이터를 운영 소스로 역병합, JAR 재빌드·개명, 소스 코드 임의 변경.

AS-IS OS 미확정 — “Rocky 7”로 단정 금지

Rocky Linux의 첫 정식 릴리스는 8.4이므로 공식적인 Rocky Linux 7은 없다. 현장에서 “7대”라고 부르는 서버는 CentOS/RHEL/Oracle Linux 등 EL7 계열일 수 있다. 작업 승인 전에 각 원본 VM에서 `/etc/os-release`, `release package`, `kernel`, `architecture`, 설치 RPM과 `native library` 의존성을 채증한다. 본 런북의 목표와 최종 합격 환경은 Oracle Linux 9.6이다.

2. 작업 원칙과 전환 모델

아주 쉽게 설명하면: 2. 작업 원칙과 전환 모델

이 부분은 이관의 기본 약속입니다.

VMware 미러는 컴퓨터 한 대를 통째로 복사해 옮기는 출발점입니다. 하지만 안에 있는 Redis·MariaDB·Solr·Elasticsearch 같은 데이터는 각각 성격이 다르기 때문에, ‘VM을 복사했으니 모든 데이터가 완벽하게 옮겨졌다’고 보면 안 됩니다.

그래서 이 런북은 큰 컴퓨터는 미러로 준비하고, 데이터는 제품이 제공하는 안전한 백업·복원 방법을 사용합니다. MariaDB는 업무 데이터라 조심하고, Redis는 다시 만들 수 있는 cache인지 확인하며, Solr와 Elasticsearch는 각각 BACKUP/RESTORE와 Snapshot/Restore를 사용합니다.

이 부분에서 확인할 것: 멈춤 기준: 원본과 target을 동시에 같은 주소로 켜지 않기, source에서 파괴 명령 실행하지 않기.



1. **미러는 운반본이다.** 미러 생성 시점이 노드별로 달라도 바이너리·설정 보존에는 쓸 수 있으나 분산 데이터의 하나의 논리 시점을 보장하지 않는다.
2. **MariaDB의 기준은 미러다.** database 전체 dump/restore를 하지 않는다. Scheduler 전용 테이블은 source 최종 data-only INSERT SQL을 만들고 target의 해당 데이터만 비운 뒤 재적재한다. 공용 업무 테이블은 관련 PK만 교정한다.
3. **Redis는 cache다.** 미러된 RDB/AOF는 그대로 사용할 수 있으나 별도 이관 원본으로 만들지 않는다. 기존 cluster가 정상 재결합하면 유지하고, 실패하면 빈 cluster를 다시 구성한 뒤 시스템 배포 기능으로 cache를 재생성한다.
4. **Solr collection 이관은 필수다.** ZooKeeper와 Solr의 미러 data를 최종 복구 원본으로 쓰지 않고, configset-alias와 모든 collection을 서비스 API의 BACKUP/RESTORE로 옮긴다.
5. **Elasticsearch는 Snapshot/Restore한다.** 미러 data directory는 증적·원복용으로 보관한 뒤 target working data를 비우고 새 cluster UUID로 구성한다. index는 snapshot으로 복원하고 ILM·SLM·template-pipeline은 export/import 후 별도 검증한다.
6. **B0와 B1의 목적이 다르다.** B0는 리허설과 복구 가능성 확인용이다. B1은 Scheduler와 모든 writer를 멈춘 뒤 만드는 Solr/ES 최종 백업 및 MariaDB Scheduler 영향 데이터의 최종 INSERT SQL이다. Redis 데이터 백업은 B1 필수 산출물이 아니다.
7. **삭제 대신 보관 후 초기화한다.** target임을 세 번 확인한 뒤 기존 동적 디렉터리를 외부 또는 hold 영역으로 이동하고, 신규 빈 경로를 만든다.

절대 안전 규칙

MariaDB의 DROP DATABASE, DROP TABLE, schema 전체 import와 전체 table TRUNCATE는 금지한다. 승인된 Scheduler 전용 테이블의 DELETE FROM은 target rollback INSERT SQL과 staging 검증을 먼저 확보한 경우에만 허용한다. Redis FLUSHALL/CLUSTER RESET과 Solr/ZK/ES data 이동은 target 식별 파일·IP·hostname을 대조하고 책임자가 서명한 뒤에만 수행한다. source에는 파괴 명령을 실행하지 않는다.

3. 현행 기준정보

** 아주 쉽게 설명하면: 3. 현행 기준정보**

이 부분은 이사 전 집의 사진입니다.

현재 어떤 서버에 어떤 프로그램이 있고, 몇 번 문(port)을 쓰며, 데이터가 어느 폴더에 있는지 기록한 표입니다. 이 표가 있어야 이사 후 '무엇이 빠졌는지'를 비교할 수 있습니다.

표의 값 중 '관찰값'과 '추정값'을 구분해야 합니다. 예를 들어 서비스 버전과 포트는 실제 조회로 확인해야 하고, 문서에 적힌 주소나 버전이 현재 실행 중인 값과 다르면 실제 조회 결과를 우선합니다.

이 부분에서 확인할 것: 확인할 것: VM별 IP·hostname·OS·서비스·포트·데이터 경로·담당자.

아래 애플리케이션·미들웨어 값은 2026-09-03 테스트베드에서 확인한 참고값이다. 운영 AS-IS OS는 EL7 계열 또는 Rocky Linux 8로 추정될 뿐 미확정이므로, 미러 작업 직전 새 manifest를 만들고 차이가 나면 실측값을 우선한다.

영역	현재 확인값	이관 시 원칙
노드	192.168.1.101/102/103, 별칭 es1-3-redis1-3-solr1-3-zk1-3 등	source용 이름과 target용 이름/IP를 분리하고 L4 연결 전 hosts/DNS를 검증
OS / Java	AS-IS EL7 계열 또는 Rocky Linux 8 추정(미확정), Java 8 계열	배포판을 실측하고 OL9.6에 승인된 JDK/JRE 8을 별도 설치해 실제 JAR·미들웨어 기동 시험. 단순 파일 존재만으로 합격 처리하지 않음
Redis	5.0.8, 7000 master ×3, 7001 replica ×3, AOF yes, 로그 /logs/redis	미러 persistence를 보존하고 cluster 정상화. 필요 시 빈 3M/3R 재구성 후 시스템 배포 기능으로 cache 재생성
ZooKeeper/Solr	ZK 3.6.1: 2181/2888/3888, Solr 7.6.0: 8983, configset chat-base-config	ZK를 빈 ensemble로 재구성하고 configset-collection backup을 복원
Solr collection	TC_CALLBOT_A/B, TC_CALLBOT_LOG, TC_CALLBOT_TEST; alias TC_CALLBOT	TC_CALLBOT_A replica가 현재 solr3에 편중되어 있어 BACKUP 성공과 복원 후 물리 분산을 필수 게이트로 지정
Elasticsearch	7.8.0, 9200/9300, cluster trusted-context-es, data /application/elk/data, 로그 /logs/elk	사전에 snapshot repository를 구성. raw data directory는 복구 원본으로 사용 금지
MariaDB	10.6.7, data /mariadb/data, 로그 /logs/maria; 테스트베드 listener 13306	실제 접속 endpoint를 변수화. 애플리케이션용 VIP가 mariadb:3306이면 L4 담당자가 별도 제공
DB engine	테스트베드의 meritx_easycms 129개 테이블이 모두 MyISAM, 그중 128개가 AUTO_INCREMENT, 실제 FK 0개로 관찰됨	전체 DB 덤프·복원은 하지 않음. Scheduler 대상 data-only 추출 때 writer 정지 + --lock-all-tables 사용, 논리 참조는 별도 SQL로 검증
애플리케이션	gateway 8081, engine 8180, cms 8280, master 8380, chat-ui 8480(/client), scheduler 8580(/scheduler)	VM에 배포된 JAR을 checksum 그대로 이관, 파일명 변경 금지, profile tc,core-tc
실행 계정	systemd 미들웨어와 애플리케이션 파일의 기본 소유자 admin:admin	애플리케이션 프로세스는 admin. root/sudo는 OS 패키지·systemd·SELinux 등 승인된 호스트 작업에만 사용

3.1 현재 VM 배포 JAR 기준 checksum

 아주 쉽게 설명하면: 3.1 현재 VM 배포 JAR 기준 checksum

checksum은 파일의 지문입니다.

JAR 파일을 복사할 때 파일 이름만 같다고 같은 파일이라는 뜻은 아닙니다. SHA-256 checksum은 파일 내용을 계산한 긴 문자열이라서, 복사 전후 값이 같으면 내용이 바뀌지 않았다고 판단하는 데 도움을 줍니다.

이 값은 ‘이 프로그램을 새로 빌드해서 교체한다’는 뜻이 아닙니다. 현재 VM에서 실제로 실행하던 JAR을 원본으로 삼아 그대로 보존하고, target에서도 같은 지문인지 확인한다는 뜻입니다.

이 부분에서 확인할 것: 성공 기준: source와 target의 파일명·크기·권한·SHA-256이 승인된 기준과 일치.

로컬 개발 PC의 target 산출물이 아니라 미리 직전 현재 VM에 배포된 파일을 원본으로 삼는다. 다음 값은 현행 참고값이며 작업 시 다시 계산한다.

모듈	파일명	SHA-256 (2026-09-03 확인)
scheduler	scheduler-0.0.1-SNAPSHOT.jar	c75fcbcfabee146d4e3f764e0320f6a20c24045537846886a2bb5b794a5af4f4d
engine	engine-0.0.1-SNAPSHOT.jar	731b172f86d0d27decb043a97511dd0c196cdd9e930b541e2d005e9f274ba931
gateway	gateway-0.0.1-SNAPSHOT.jar	9058ff645a40415e5d8af15011f3377b642e1592f02635eebe76670c0788dd6c
cms	cms-0.0.1-SNAPSHOT.jar	477fb0464bb5976c001e19f1ce11737c4b0da377611b35690cc1d33be495a9e2
master	master-0.0.1-SNAPSHOT.jar	9f6bc9c95f9e2907130dbd35b189007485010cbe326720089ac347debd155ae4
chat-ui	chat-ui-0.0.1-SNAPSHOT.jar	e3f1450f237600d440a6e3330524f86545c2acc5b4845e26457b846544f36104

파일성 업무 데이터

CMS 업로드 파일은 노드별 내용이 다를 수 있다. 각 VM의 업로드 경로를 노드명과 함께 별도 보관하고 checksum manifest를 만든다. 서로 덮어쓰거나 무조건 합치지 말고 DB 참조와 대조한 뒤 승인된 파일만 최종 배치한다.

4. 역할·사전조건·변수

아주 쉽게 설명하면: 4. 역할·사전조건·변수

이 부분은 누가 무엇을 준비해야 하는지 정하는 곳입니다.

서버 이관은 한 사람이 명령어를 잘 입력한다고 끝나지 않습니다. VMware 담당자는 VM을 옮길 수 있어야 하고, DB 담당자는 데이터가 맞는지 판단해야 하며, 애플리케이션 담당자는 CMS와 시뮬레이터가 정상인지 확인해야 합니다.

변수 시트는 명령어 속 빈칸을 실제 값으로 채우는 표입니다. 예를 들어 SOURCE_IP, TARGET_IP, VM UUID, SR UUID를 잘못 넣으면 엉뚱한 서버를 건드릴 수 있으므로, 명령어보다 먼저 변수표를 확정합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: 작업자·승인자·Xen 관리자·DB 담당자·애플리케이션 담당자의 연락처, 실제 주소·UUID·백업 경로입니다.

실제 흐름: 누가 해도 된다고 승인하는지, 누가 결과를 판단하는지, 누가 source로 돌릴지를 적습니다. IP·UUID를 추측해서 변수표를 채우지 않습니다.

1. 사람의 역할을 정합니다.
2. 서버·저장소 식별자를 확정합니다.
3. 승인 게이트를 정합니다.
4. 실행 기록 양식을 준비합니다.

멈춤 조건: 담당자·target 주소·백업 경로·rollback 방법 중 하나라도 없으면 명령을 칠 수 있어도 시작하지 않습니다.

이 부분에서 확인할 것: 멈춤 기준: 대상 IP·VM UUID·저장소·담당자가 하나라도 미확정이면 실행하지 않음.

역할	책임	필수 승인/산출물
작업책임자	전체 GO/NO-GO, writer freeze, 파괴 작업 승인, cutover/rollback 결정	단계별 서명, 업무 중단 시각
VMware/OS 담당	AS-IS OS 실측, 일관성 그룹·guest quiesce 설정, 격리 포트그룹, OL9.6 전환, 원복 가능한 원본 보존	OS manifest, VM snapshot/clone ID, 시작·종료 시각
DB 담당	미러 DB 검사, Scheduler 영향 테이블/PK의 data-only SQL 추출·원복·재적재, GTID/binlog 좌표, DB 솔루션 연계	대상 목록, INSERT SQL checksum, 전후 count/checksum, rollback SQL
미들웨어 담당	Redis 미러 cluster 판정·cache 재배포, Solr/ZK 재구성과 collection 이관, ES snapshot·정책 복원·장애 시험	cluster manifest, collection/snapshot/policy 성공 증적
애플리케이션 담당	JAR/config checksum, endpoint 격리, smoke/E2E, Scheduler 단일 기동	테스트 결과, 로그
L4/네트워크 담당	target health check 등록, VIP 전환·원복, source/target 동시 노출 방지	pool 상태, 전환 시각

4.1 작업 전 필수조건

아주 쉽게 설명하면: 4.1 작업 전 필수조건

이 목록은 출발 신호가 아니라 안전벨트입니다.

백업이 있고, target 저장공간이 충분하고, 테스트 IP가 있고, 롤백 담당자가 정해져 있어야 작업을 시작할 수 있다는 뜻입니다. 하나라도 빠지면 문제 발생 시 멈추거나 되돌릴 수 없습니다.

특히 '운영 IP와 다른 임시 IP', 'Scheduler를 한 대만 켜 방법', 'source와 target을 분리할 네트워크'가 중요합니다.

이 부분에서 확인할 것: GO 조건: 모든 체크박스 완료와 담당자 승인. NO-GO 조건: 한 항목이라도 미확정.

- ☐ 소스와 대상이 동시에 같은 IP/hostname/VIP로 통신하지 않는 격리 설계가 승인됐다.
 - ☐ 미러 디스크와 독립된 백업 저장소가 있고 용량·권한·checksum 복사가 검증됐다.
- Solr 세 노드에서 같은 경로로 보이는 공유 BACKUP 저장소가 준비됐다.

- ☐ Elasticsearch 세 노드의 path.repo와 snapshot repository가 미리 준비되고 verify에 성공했다.
- ☐ 운영의 master DB와 모든 tenant DB, 실제 engine, 전체 용량, 가장 큰 테이블, dump/restore 예상 시간이 조사됐다.
- ☐ MariaDB binlog 보존기간이 리허설 기간보다 길다. 현재 테스트베드 관찰값 7일을 운영값으로 가정하지 않는다.
- ☐ JDK/JRE 8 설치 매체와 Oracle Linux 9.6용 필수 native library 패키지가 내부 저장소에 준비됐다.
- ☐ 비밀값은 vault 또는 권한 0600 option file로 제공하고 명령행·문서·로그에 기록하지 않는다.

4.2 변수 시트

아주 쉽게 설명하면: 4.2 변수 시트

변수 시트는 작업용 주소록입니다.

긴 명령어를 입력할 때 매번 주소와 UUID를 직접 타이핑하면 실수하기 쉽습니다. 이 표에 원본 주소, target 주소, 포트, VM UUID, SR UUID, 백업 경로를 적고, 명령어의 빈칸에 넣습니다.

이 표는 단순 메모가 아니라 '이번 작업에서 어느 대상을 사용할 것인지'를 고정하는 기준입니다. 작업 중 주소가 바뀌면 기존 값을 지우지 말고 변경 이유와 승인자를 기록해야 합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: 작업자·승인자·Xen 관리자·DB 담당자·애플리케이션 담당자의 연락처, 실제 주소·UUID·백업 경로입니다.

실제 흐름: 누가 해도 된다고 승인하는지, 누가 결과를 판단하는지, 누가 source로 돌릴지를 적습니다. IP·UUID를 추측해서 변수표를 채우지 않습니다.

1. 사람의 역할을 정합니다.
2. 서버·저장소 식별자를 확정합니다.
3. 승인 게이트를 정합니다.
4. 실행 기록 양식을 준비합니다.

멈춤 조건: 담당자·target 주소·백업 경로·rollback 방법 중 하나라도 없으면 명령을 칠 수 있어도 시작하지 않습니다.

이 부분에서 확인할 것: 확인할 것: 실제 xen03 주소(.120인지 .130인지), source/target IP, 백업 저장소, 실행 계정.

```

RUN_ID=<YYYYMMDDThhmmss>
BACKUP_ROOT=/migration-backup/${RUN_ID}      # 미러 디스크 밖의 공유/이관 저장소
HOLD_ROOT=/migration-hold/${RUN_ID}          # 대상 기존 상태 보관
MASTER_DB_NAME=<master schema; testbed=meritz_easycms>
TENANT_DB_NAME=<tenant schema; repeat for every active tenant>

SRC_DB_HOST=<source DB/VIP>
SRC_DB_PORT=<source listener>
TGT_DB_HOST=<isolated target DB endpoint>
TGT_DB_PORT=<target listener>

SRC_REDIS_SEED=<source redis host:port>
TGT_REDIS_SEED=<target redis1:7000>
SRC_SOLR_URL=http://<source-solr>:8983
TGT_SOLR_URL=http://<target-solr>:8983
SRC_ZK=<source-zk1:2181,source-zk2:2181,source-zk3:2181>
TGT_ZK=zk1:2181,zk2:2181,zk3:2181
SRC_ES_URL=http://<source-es>:9200
TGT_ES_URL=http://<target-es>:9200

```

값을 채운 변수 시트는 source용과 target용을 색상·파일명으로 분리한다. 운영 비밀번호와 암호화 키는 이 시트에 쓰지 않는다.

5. G0 — 사전 채증과 B0 리허설 백업

아주 쉽게 설명하면: 5. G0 — 사전 채증과 B0 리허설 백업

G0는 출발 전에 사진을 찍고 연습용 백업을 만드는 단계입니다.

채증은 현재 상태를 증거로 남기는 일입니다. 어떤 서비스가 살아 있고, 어떤 데이터가 몇 건이며, 어느 파일을 실행하는지 기록합니다. B0는 운영 전환 직전의 최종 백업이 아니라, 먼저 target에서 복원 연습을 해 보기 위한 백업입니다.

이 단계에서 서버를 바꾸는 것이 아니라 현재 상태를 읽고 저장합니다. 나중에 target 결과와 비교할 수 있도록 명령어 결과, checksum, 서비스 상태, 데이터 목록을 파일로 남깁니다.

조금 더 자세하: 실제 작업의 흐름

들어오는 자료: 원본의 현재 상태, 백업 저장 위치, 생성 시각, checksum, 복원 방법입니다. 백업은 파일 한 개를 복사하는 행동이 아니라 실패했을 때 돌아갈 지점을 만드는 일입니다.

B0와 B1: B0는 리허설 전 복원 연습용이고, B1은 최종 전환 직전 writer와 Scheduler를 멈춘 뒤 마지막 변경분을 고정하는 용도입니다. 이름이 비슷해도 시점과 목적이 다릅니다.

1. 백업 대상을 정합니다.
2. 파일·시각·크기를 기록합니다.
3. SHA-256으로 변조 여부를 봅니다.
4. target에서 실제 복원합니다.
5. 성공·실패와 승인을 남깁니다.

초보자 원칙: '백업 성공' 메시지만 믿지 말고 목록·checksum·복원 결과까지 확인합니다.

이 부분에서 확인할 것: 다음 단계로 가는 조건: B0 백업이 실제로 복원 가능한지 확인하고, 원본 변경 없이 기록 완료.

B0는 target 복원 절차와 Oracle Linux 9.6 호환성을 시험하기 위한 기준본이다. MariaDB 전체 dump는 만들지 않으며 Scheduler 전용 데이터와 공용 상태 후보만 선택 추출한다. Solr collection backup과 Elasticsearch snapshot의 성공을 확인하지 못하면 미러 단계로 넘어가지 않는다.

5.1 공통 현황 채증

아주 쉽게 설명하면: 5.1 공통 현황 채증

각 서버의 건강검진표를 만드는 일입니다.

CPU·메모리·디스크·OS·프로세스·포트·로그·설정 주소를 조사합니다. 의사가 환자의 체온과 혈압을 먼저 재듯이, 이관 전에 서버의 현재 모습을 먼저 알아야 합니다.

명령어의 목적은 '무언가를 고치는 것'이 아니라 '현재 상태를 읽는 것'입니다. 결과를 서버별 파일로 저장하면 101·102·103을 헛갈리지 않고 target과 비교할 수 있습니다.

이 부분에서 확인할 것: 성공 기준: 세 VM의 hostname, IP, OS, 서비스, 포트, 디스크와 연결 대상이 표로 정리됨.

- ☐ 각 VM의 hostname, IP, OS/kernel, CPU architecture, 시간대/NTP, filesystem/mount, free space.
- ☐ admin의 numeric UID/GID, 디렉터리 ownership/mode, SELinux context, file capabilities.
- ☐ Java 경로·버전, 환경변수, 모든 native executable의 version과 ldd 결과.
- ☐ systemd unit 전체 내용·enable 상태·실행 계정, 애플리케이션 appctl.sh 및 실제 JAR 파일명.
- ☐ /application, /mariadb, 업무 파일 디렉터리의 파일 목록·크기·SHA-256.
- ☐ listen port, 방화벽, hosts/DNS, L4 health URL, 각 서비스 클러스터 토폴로지와 데이터 건수.
- ☐ 설정 파일은 백업하되 DB 암호, Jasypt 키, API key 등은 증적본에서 마스킹.

5.2 MariaDB B0 선택 데이터 추출

아주 쉽게 설명하면: 5.2 MariaDB B0 선택 데이터 추출

MariaDB에서 필요한 일부 데이터의 연습용 복사본을 만드는 단계입니다.

MariaDB에는 실제 업무 데이터가 있을 수 있으므로 데이터베이스 전체를 무작정 비우거나 복원하지 않습니다. Scheduler가 만드는 통계·배치 기록처럼 다시 넣어야 하는 범위만 골라서 INSERT SQL로 준비합니다.

B0는 연습용입니다. source의 데이터가 언제 기준인지 기록하고, target에 적용하기 전에 행 수·기본키·checksum을 비교합니다. MyISAM 테이블은 일반적인 transaction 되돌리기가 제한될 수 있어 원복 SQL을 별도로 준비해야 합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: source DB의 현재 데이터, target DB의 table 구조, Scheduler가 건드리는 table 목록, 그리고 승인된 적용 범위입니다. DB는 단순한 파일 창고가 아니라 여러 프로그램이 계속 읽고 쓰는 업무 장부이므로, '파일이 복사됐나'보다 '어떤 행이 같은가'를 확인합니다.

실제 흐름: 먼저 읽기 명령으로 접속·table·행 수·진행 중 작업을 확인합니다. 그다음 writer와 Scheduler를 멈추고 source에서 승인된 data-only INSERT SQL을 만듭니다. target에서는 승인된 범위만 preclear하고 INSERT한 뒤 source와 count·기본키·checksum을 비교합니다. 나머지 업무 원본 table은 건드리지 않는 것이 기본입니다.

1. **읽기:** DB가 살아 있고 구조가 맞는지 봅니다.
2. **고정:** 더 이상 행이 변하지 않도록 writer를 멈춥니다.
3. **적용:** A그룹 전량 또는 B그룹 후보 PK만 처리합니다.
4. **비교:** 다르면 성공으로 보지 않고 멈춥니다.

가장 중요한 이유: MyISAM에서는 실패하면 DB가 자동으로 원래 상태로 돌아온다고 가정하면 안 됩니다. 따라서 지우기 전에 before-image와 원복 INSERT SQL을 먼저 준비합니다.

이 부분에서 확인할 것: 금지: source에서 DROP/TRUNCATE, 승인 없는 전체 DB dump/restore.

MyISAM 주의

현재 테스트베드는 모든 업무 테이블이 MyISAM이다. --single-transaction은 일관성을 보장하지 않는다. 선택 데이터 추출 중에도 Scheduler와 관련 writer를 멈추고 --lock-all-tables로 일관된 시점을 잡는다. database 전체 dump-restore는 본 절차의 대상이 아니다.

모든 활성 tenant schema 조사

코드는 master DB의 tbl_manage_tenant.manage_schema_id → tbl_manage_schema 관계에서 tenant별 접속정보를 읽어 DataSource를 동적으로 만든다. 아래 Scheduler 전용 통계 테이블은 각 활성 tenant DB에 있을 수 있으므로 master DB 한 곳만 보고 끝내지 않는다. 서로 다른 DB 서버라면 각 서버의 writer를 멈춘 시각과 추출 완료시각을 기록한다.

```
SELECT mt.code, mt.name, ms.prefix_url, ms.schema_name, ms.user_name
FROM tbl_manage_tenant mt
JOIN tbl_manage_schema ms ON ms.id = mt.manage_schema_id
WHERE COALESCE(mt.is_del, 'N') = 'N';
```

1. Scheduler를 먼저 중지하고 신규 배포·학습·통계 job이 시작되지 않음을 확인한다.
2. 공용 예약 배포 상태까지 추출할 때는 CMS/Master 등 관련 writer도 잠시 차단한다.
3. 암호는 권한 0600인 <SOURCE_DB_OPTION_FILE>에서 읽는다.
4. master DB에서는 tbl_batch_log, 각 tenant DB에서는 9개 통계 table만 data-only·명시 컬럼 INSERT 형식으로 추출한다. 테스트베드처럼 master/tenant가 같은 schema일 때만 10개를 한 파일로 합친다. DDL, 사용자, GRANT, system DB는 포함하지 않는다.

```
mariadb-dump \
--defaults-extra-file=<SOURCE_DB_OPTION_FILE> \
--host="${SRC_DB_HOST}" --port="${SRC_DB_PORT}" \
--no-create-info --complete-insert --skip-extended-insert \
--skip-triggers --quick --lock-all-tables --hex-blob \
--skip-add-locks --skip-disable-keys --skip-dump-date \
"${TENANT_DB_NAME}" \
tbl_api_statistics tbl_dialog_statistics \
tbl_knowledge_statistics tbl_satisfaction_statistics \
tbl_scenario_statistics tbl_scenario_node_statistics \
tbl_slot_statistics tbl_top10_dialog_statistics tbl_user_statistics \
> "${BACKUP_ROOT}/mariadb/${TENANT_DB_NAME}.B0.statistics.insert.sql"

mariadb-dump \
--defaults-extra-file=<SOURCE_DB_OPTION_FILE> \
--host="${SRC_DB_HOST}" --port="${SRC_DB_PORT}" \
--no-create-info --complete-insert --skip-extended-insert \
--skip-triggers --quick --lock-all-tables --hex-blob \
--default-character-set=utf8mb4 --order-by-primary \
--skip-add-locks --skip-disable-keys --skip-dump-date \
"${MASTER_DB_NAME}" tbl_batch_log \
> "${BACKUP_ROOT}/mariadb/${MASTER_DB_NAME}.B0.batch-log.insert.sql"

sha256sum "${BACKUP_ROOT}/mariadb/"*.B0.*.insert.sql
```

각 table의 정확한 row count, CHECKSUM TABLE, PK min/max, AUTO_INCREMENT와 dump exit code를 manifest에 기록한다. tbl_learn_recommend, tbl_deploy, tbl_manage_deploy, tbl_bot은 공용 업무 상태이므로 위 전체 추출 목록에 넣지 않고 Scheduler 실행-예약 배포와 연관된 PK만 별도 후보 SQL로 보관한다.

5.3 Redis B0 상태 채증

아주 쉽게 설명하면: 5.3 Redis B0 상태 채증

Redis cluster의 자리 배치표를 만드는 단계입니다.

Redis는 여러 대가 master와 replica로 나뉘고, 16,384개의 slot에 데이터를 나누어 맡길 수 있습니다. 따라서 'Redis 프로세스가 켜져 있다'보다 어느 노드가 어떤 역할인지와 모든 slot이 정상인지가 중요합니다.

RDB-AOF-cluster state 파일은 바로 새 cluster에 덮어쓰지 않고 hold 영역에 보관합니다. target에서 기존 cluster를 재결합할지, 빈 cluster를 만들고 cache를 다시 생성할지 결정할 수 있도록 상태를 기록합니다.

조금 더 자세하: 실제 작업의 흐름

들어오는 자료: Redis 노드의 주소·port·master/replica 역할·node ID·slot 상태와 AOF/RDB 파일 정보입니다. Redis가 cache만 저장하는지, 로그인 세션·queue·업무 상태도 저장하는지는 애플리케이션 설정으로 확인합니다.

실제 흐름: PONG은 한 노드가 대답했다는 뜻이고, cluster_state:ok와 모든 slot 정상은 집단 전체가 일할 준비가 됐다는 뜻입니다. 기존 cluster 재결합이 안전하면 유지하고, 주소·node ID·slot이 꼬이면 빈 3 master/3 replica를 구성한 뒤 cache를 다시 만듭니다.

1. 노드와 역할을 기록합니다.
2. source와 target이 동시에 같은 cluster가 되지 않게 합니다.
3. 재결합 또는 새 cluster 중 하나를 선택합니다.
4. cache 재생성 뒤 TTL·대표 기능을 확인합니다.

주의: 'Redis니까 지워도 된다'가 아닙니다. cache라면 다시 만들 수 있지만 세션·queue라면 초기화가 곧 업무 영향일 수 있으므로 담당자 승인이 필요합니다.

이 부분에서 확인할 것: 성공 기준: master/replica-slot·node ID·AOF/RDB 경로와 checksum 기록.

1. 7000이 master라고 가정하지 말고 CLUSTER NODES와 각 노드의 INFO replication으로 현재 master 세 개를 식별한다.
2. 모든 7000/7001의 host:port, node ID, master/replica 관계, slot, DBSIZE, INFO keyspace, dir, RDB/AOF와 자동 cluster state 파일 경로·checksum을 기록한다.
3. RDB/AOF는 VMware 미러본과 함께 비상 원복본으로 보관할 수 있으나 Redis 데이터의 별도 B0/B1 이관 산출물로 요구하지 않는다.

```
/application/redis/bin/redis-cli -h "${REDIS_HOST}" -p "${REDIS_PORT}" CLUSTER NODES
/application/redis/bin/redis-cli -h "${REDIS_HOST}" -p "${REDIS_PORT}" CLUSTER INFO
/application/redis/bin/redis-cli -h "${REDIS_HOST}" -p "${REDIS_PORT}" INFO persistence
/application/redis/bin/redis-cli -h "${REDIS_HOST}" -p "${REDIS_PORT}" INFO replication
```

AOF가 활성화된 인스턴스는 시작할 때 RDB보다 AOF를 우선 사용할 수 있으므로 RDB만 교체하거나 삭제하지 않는다. 검사 도구의 --fix는 원본에 실행하지 않는다.

5.4 Solr/ZooKeeper B0 백업

아주 쉽게 설명하면: 5.4 Solr/ZooKeeper B0 백업

검색 설계도와 검색 문서를 연습용으로 보관하는 단계입니다.

ZooKeeper는 SolrCloud가 어느 서버와 collection을 사용할지 조정하는 역할이고, Solr collection은 실제 검색 문서 묶음입니다. 둘 중 하나만 옮기면 화면은 떠도 검색이 실패할 수 있습니다.

configset은 검색 규칙과 필드 설계도이고, collection backup은 실제 문서입니다. 모든 collection과 alias를 목록으로 만든 뒤 collection별 BACKUP이 완료됐는지 확인합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: ZooKeeper ensemble 주소와 myid, Solr node 주소, configset, collection 목록, shard-replica-leader-alias 정보입니다. ZooKeeper는 SolrCloud의 조정자이고 Solr는 실제 검색 문서를 제공하므로 둘을 한 세트로 봅니다.

실제 흐름: 새 ZooKeeper가 quorum을 만든 뒤 검색 규칙인 configset을 올립니다. 그다음 source의 모든 collection을 하나씩 BACKUP/RESTORE합니다. 복원 후에는 이름만 보지 않고 leader-replica-alias-문서 수·대표 한국어 검색을 확인합니다.

1. 새 ZK ensemble을 준비합니다.
2. Solr가 새 ZK를 보게 합니다.
3. configset과 collection을 복원합니다.
4. shard-replica-leader-alias-검색을 검사합니다.

자주 하는 착각: Solr 8983 port가 열려 있어도 collection이나 leader가 없으면 검색 시스템은 완성되지 않은 것입니다. ZK의 ruok 응답도 프로세스 생존 확인일 뿐 quorum 전체 정상 판정은 아닙니다.

이 부분에서 확인할 것: 성공 기준: source collection 목록과 backup 완료 목록이 하나도 빠짐없이 일치.

1. CLUSTERSTATUS, live node, collection/shard/replica/leader/configName, alias, collection별 numFound, configset, ktnori JAR 위치/checksum을 저장한다.
2. 각 collection에 hard commit을 실행한다. soft commit으로만 보이는 문서는 BACKUP에 포함되지 않을 수 있다.

3. 세 Solr 노드에 같은 경로로 mount된 저장소에 collection별 BACKUP을 비동기로 실행하고 REQUESTSTATUS=completed를 확인한다.
4. chat-base-config를 solr zk downconfig로 별도 저장하고, 존재하는 security.json, clusterprops.json, aliases.json, roles.json을 별도 export한다.

```
curl -fsS -X POST \  
  "${SRC_SOLR_URL}/solr/${COLLECTION}/update?waitSearcher=true" \  
  -H "Content-Type: application/json" --data-binary '{"commit":{}}'  
  
curl -fsSG "${SRC_SOLR_URL}/solr/admin/collections" \  
  --data-urlencode "action=BACKUP" \  
  --data-urlencode "collection=${COLLECTION}" \  
  --data-urlencode "name=${BACKUP_NAME}" \  
  --data-urlencode "location=${SOLR_BACKUP_LOCATION}" \  
  --data-urlencode "async=${REQUEST_ID}" --data-urlencode "wt=json"
```

기존 ZooKeeper의 version-2는 장애 분석용으로만 보관한다. 신규 ensemble에 복원하지 않는다. 특히 TC_CALLBOT_A BACKUP 성공과 checksum이 없으면 G0 실패다.

5.5 Elasticsearch B0 snapshot

아주 쉽게 설명하면: 5.5 Elasticsearch B0 snapshot

Elasticsearch의 검색 데이터를 공식 백업 방식으로 찍는 단계입니다.

Snapshot은 Elasticsearch가 자신의 index와 shard 구조를 이해한 상태로 저장하는 백업입니다. 실행 중인 data 폴더를 파일 복사하는 것보다 cluster가 읽을 수 있는 형식으로 복원하기 쉽습니다.

repository는 Snapshot을 넣어 둘 공용 창고이고, B0 snapshot은 target에서 복원 연습을 위한 백업입니다. index뿐 아니라 template·pipeline·ILM·SLM 정책을 별도로 확인해야 합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: Elasticsearch node-cluster 정보, index·shard·alias 목록, Snapshot repository, template·pipeline·ILM·SLM 정책입니다. Elasticsearch는 index를 여러 shard로 나누어 저장하므로 data 폴더를 일반 파일처럼 복사하지 않습니다.

실제 흐름: target에서 새 빈 cluster를 준비하고 repository를 등록한 뒤 성공한 Snapshot을 restore합니다. 복원 후 cluster health·node-index·shard를 보고, 정책 객체는 별도로 비교합니다. green은 shard 배치가 정상이라는 뜻이지 업무 검색까지 자동 보장한다는 뜻은 아닙니다.

1. 새 cluster와 repository를 준비합니다.
2. 필요한 snapshot과 index를 복원합니다.
3. template·pipeline·ILM·SLM을 따로 확인합니다.
4. 대표 검색·정렬·aggregation·재기동을 시험합니다.

raw data 폴더를 피하는 이유: 그 안에는 cluster UUID와 shard 상태 같은 내부 정보가 들어 있어 버전·노드·시점이 맞지 않으면 cluster가 깨질 수 있습니다.

이 부분에서 확인할 것: 성공 기준: snapshot SUCCESS, 실패 shard 0, 예상 index와 정책 목록 기록.

현재 테스트베드에는 snapshot repository가 없는 것으로 확인됐다. 미러 전 세 노드에 공유 repository와 path.repo를 준비하고 _verify에 성공해야 한다. 이 정적 설정에 rolling restart가 필요하면 별도 사전 작업으로 승인받는다.

```
PUT /_snapshot/ol_migration_repo
{
  "type": "fs",
  "settings": {
    "location": "/migration-backup/elasticsearch",
    "compress": true
  }
}

POST /_snapshot/ol_migration_repo/_verify

PUT /_snapshot/ol_migration_repo/<B0_SNAPSHOT>?wait_for_completion=true
{
  "indices": "<EXACT_APPLICATION_KIBANA_AND_OPTIONAL_SECURITY_INDEX_LIST>",
  "ignore_unavailable": false,
  "include_global_state": true,
  "partial": false,
  "metadata": { "purpose": "OL migration rehearsal", "run_id": "<RUN_ID>" }
}
```

합격값은 state=SUCCESS, failed shard 0이며 예상 index가 snapshot 목록에 전부 존재해야 한다. cluster UUID/name, node/role, health, cat indices/shards, index별 count/mapping/settings/alias를 저장한다. 또한 _cluster/settings, _template, _index_template, _component_template, _ingest/pipeline, _ilm/policy, _slm/policy, _snapshot, _ilm/status, _slm/status 응답을 각각 JSON으로 export한다. repository 내부 파일은 직접 편집하지 않는다.

GO — 미리 착수 승인

- ☐ MariaDB Scheduler 전용 data-only INSERT SQL-공용 상태 후보·count/checksum manifest 확보
- ☐ Redis 여섯 노드의 cluster topology와 RDB/AOF/cluster-state 경로-checksum 채증
- ☐ 모든 Solr collection BACKUP completed, configset-alias-count manifest 확보 — 하나라도 누락되면 NO-GO
- ☐ ES snapshot SUCCESS, failed shard 0, repository verify 및 ILM·SLM·template·pipeline export 성공
- ☐ JAR/config/upload 파일 checksum과 서비스/포트 현황 확보

책임자: _____ 시각: _____

6. M0 — VMware 미러 생성·인수

아주 쉽게 설명하면: 6. M0 — VMware 미러 생성·인수

VM 한 대를 통째로 복사해 새 장소에 가져오는 단계입니다.

VM은 가상 컴퓨터입니다. VMware 미러에는 보통 가상 디스크 안의 OS와 설치된 프로그램, 설정 파일이 함께 들어갑니다. 그래서 OS를 새로 설치하는 것보다 현재 상태를 빠르게 재현할 수 있습니다.

하지만 미러는 '안전하게 운영할 준비가 끝났다'는 뜻이 아닙니다. 복사본은 원본과 같은 IP·hostname·cluster 정보를 가질 수 있으므로, target에서는 반드시 격리된 상태로 첫 부팅을 합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: source VM의 CPU·RAM·디스크·NIC·OS·서비스와 target host의 저장공간·네트워크·자원입니다. VM은 가상 CPU·메모리·디스크·네트워크 카드를 가진 컴퓨터입니다.

실제 흐름: 가상화 담당자가 VM을 복사하거나 export/import합니다. 복사본을 처음 켤 때는 원본과 다른 임시 IP·hostname을 사용합니다. 부팅 후 OS·mount·/data·자동기동 서비스·Scheduler 상태를 확인합니다.

1. source 상태와 백업을 고정합니다.
2. target 저장소와 network를 확인합니다.
3. VM을 복사·인수합니다.
4. 임시 주소로 격리 부팅합니다.
5. 원본과 충돌이 없는지 확인합니다.

비유: VM 복사는 집 전체를 새 동네에 복사하는 일입니다. 주소판과 전화번호가 예전 집과 같으면 우편과 전화가 섞이므로, 새 주소를 붙이고 손님을 받기 전까지 격리합니다.

이 부분에서 확인할 것: 인수 기준: VM 이름·UUID·디스크·NIC·전원 상태 확인, 원본 보존.

6.1 VMware 담당 요청사항

아주 쉽게 설명하면: 6.1 VMware 담당 요청사항

VMware 담당자에게 정확히 부탁하는 목록입니다.

이관 담당자가 직접 모든 가상화 설정을 알아서 바꾸는 것이 아니라, VM을 복사할 사람에게 필요한 조건을 전달합니다. CPU·메모리·디스크·네트워크·스냅샷·임시 IP 조건을 명확하게 요청해야 합니다.

특히 미리 생성 시점과 대상 위치를 기록해야 합니다. 세 VM이 서로 다른 시각에 복사되면 데이터가 하나의 같은 순간을 나타내지 않을 수 있으므로, 그 시각을 나중에 최종 동기화 계획에서 고려합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: source VM의 CPU·RAM·디스크·NIC·OS·서비스와 target host의 저장공간·네트워크·자원입니다. VM은 가상 CPU·메모리·디스크·네트워크 카드를 가진 컴퓨터입니다.

실제 흐름: 가상화 담당자가 VM을 복사하거나 export/import합니다. 복사본을 처음 켤 때는 원본과 다른 임시 IP·hostname을 사용합니다. 부팅 후 OS·mount·/data·자동기동 서비스·Scheduler 상태를 확인합니다.

1. source 상태와 백업을 고정합니다.
2. target 저장소와 network를 확인합니다.
3. VM을 복사·인수합니다.
4. 임시 주소로 격리 부팅합니다.
5. 원본과 충돌이 없는지 확인합니다.

비유: VM 복사는 집 전체를 새 동네에 복사하는 일입니다. 주소판과 전화번호가 예전 집과 같으면 우편과 전화가 섞이므로, 새 주소를 붙이고 손님을 받기 전까지 격리합니다.

이 부분에서 확인할 것: 확인할 것: target host, datastore/SR, network, VM 중지 시간, snapshot 보존 기간.

- ☐ 가능하면 3 VM을 같은 consistency group으로 처리하고 VMware Tools guest quiescing을 사용한다.
- ☐ 메모리 상태를 복제하지 않는다. 대상은 전원이 꺼진 상태, NIC disconnected 또는 격리 포트그룹으로 인계한다.
- ☐ 독립 디스크/RDM/외부 mount가 미리 범위에서 빠지는지 사전에 명시한다.
- ☐ VM별 snapshot/clone ID, 복제 시작·종료 시각, quiesce 성공 여부를 인계서에 남긴다.
- ☐ source VM은 최종 cutover/rollback 종료 전 삭제·업그레이드·재구성하지 않는다.

Hot mirror의 한계

사용량이 0인 시점이라도 DB buffer, AOF, Lucene segment, ZooKeeper epoch와 세 VM의 복제 시점은 다를 수 있다. 사용량 0은 위험을 낮출 뿐 서비스 단위 snapshot을 대체하지 않는다.

6.2 최초 target 부팅

아주 쉽게 설명하면: 6.2 최초 target 부팅

새로 가져온 복사본을 처음 켜 보는 단계입니다.

첫 부팅의 목적은 서비스 사용자에게 공개하는 것이 아니라, VM이 부팅되고 디스크가 보이며 네트워크와 OS가 정상인지 확인하는 것입니다. 원본과 같은 IP로 바로 켜면 두 컴퓨터가 같은 주소를 사용해 충돌합니다.

따라서 임시 IP 또는 격리 네트워크를 사용하고, Scheduler와 자동기동 서비스가 갑자기 실행되지 않았는지 확인합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: source VM의 CPU·RAM·디스크·NIC·OS·서비스와 target host의 저장공간·네트워크·자원입니다. VM은 가상 CPU·메모리·디스크·네트워크 카드를 가진 컴퓨터입니다.

실제 흐름: 가상화 담당자가 VM을 복사하거나 export/import합니다. 복사본을 처음 켤 때는 원본과 다른 임시 IP·hostname을 사용합니다. 부팅 후 OS·mount·/data·자동기동 서비스·Scheduler 상태를 확인합니다.

1. source 상태와 백업을 고정합니다.
2. target 저장소와 network를 확인합니다.
3. VM을 복사·인수합니다.
4. 임시 주소로 격리 부팅합니다.
5. 원본과 충돌이 없는지 확인합니다.

비유: VM 복사는 집 전체를 새 동네에 복사하는 일입니다. 주소판과 전화번호가 예전 집과 같으면 우편과 전화가 섞이므로, 새 주소를 붙이고 손님을 받기 전까지 격리합니다.

이 부분에서 확인할 것: 절대 하지 않기: 원본과 target의 동일 IP 동시 기동, target의 즉시 L4 등록.

1. 운영망과 분리된 포트그룹에서 한 대씩 부팅한다. source와 동일 IP가 라우팅되지 않아야 한다.
2. 모든 애플리케이션, Scheduler, MariaDB, Redis, ZooKeeper, Solr, Elasticsearch, Kibana의 자동기동을 차단한 상태인지 확인한다.
3. 대상 식별 파일을 각 VM에 만들고 변경 작업 전 항상 파일 내용·hostname·target IP를 대조한다.
4. 미러 직후의 /application, /mariadb, /logs는 원본 상태 보관본으로 snapshot/hold 처리한다.
5. 새 IP/hostname/hosts/DNS를 적용하되 L4 pool에는 아직 넣지 않는다.

7. R0 — Oracle Linux 9.6 대상 준비

아주 쉽게 설명하면: 7. R0 — Oracle Linux 9.6 대상 준비

target 운영체제와 실행 환경을 준비하는 단계입니다.

목표는 Oracle Linux 9.6에서 기존 애플리케이션이 실행되는 것입니다. VM 미러를 사용하더라도 OS가 무엇인지, Java·Python·라이브러리·권한이 맞는지 확인해야 합니다.

이 단계는 애플리케이션 소스 코드를 새로 개발하는 단계가 아닙니다. 기존 파일을 보존하고, 새 OS에서 실행에 필요한 기반 환경을 맞추는 단계입니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: source VM의 CPU·RAM·디스크·NIC·OS·서비스와 target host의 저장공간·네트워크·자원입니다. VM은 가상 CPU·메모리·디스크·네트워크 카드를 가진 컴퓨터입니다.

실제 흐름: 가상화 담당자가 VM을 복사하거나 export/import합니다. 복사본을 처음 켤 때는 원본과 다른 임시 IP·hostname을 사용합니다. 부팅 후 OS·mount·/data·자동기동 서비스·Scheduler 상태를 확인합니다.

1. source 상태와 백업을 고정합니다.
2. target 저장소와 network를 확인합니다.
3. VM을 복사·인수합니다.
4. 임시 주소로 격리 부팅합니다.
5. 원본과 충돌이 없는지 확인합니다.

비유: VM 복사는 집 전체를 새 동네에 복사하는 일입니다. 주소판과 전화번호가 예전 집과 같으면 우편과 전화가 섞이므로, 새 주소를 붙이고 손님을 받기 전까지 격리합니다.

이 부분에서 확인할 것: 성공 기준: OS·커널·architecture·JDK·라이브러리·파일 권한이 승인 기준과 일치.

한 번에 바꾸는 변수는 OS뿐

첫 리허설은 Redis 5.0.8, Solr 7.6.0, ZooKeeper 3.6.1, Elasticsearch/Kibana 7.8.0, MariaDB 10.6.7, Java 8과 기존 JAR을 유지한다. OS 전환과 미들웨어 업그레이드를 동시에 하지 않는다. 제품 버전 업그레이드는 OL9.6 기동·복원 시험이 끝난 뒤 별도 프로젝트로 수행한다.

7.1 AS-IS 배포판 판정 게이트

아주 쉽게 설명하면: 7.1 AS-IS 배포판 판정 게이트

원본 OS 이름을 추측하지 않고 확인하는 문입니다.

‘EL7 계열’은 RHEL, CentOS, Oracle Linux 같은 Red Hat 계열의 7 버전대라는 뜻일 수 있습니다. Rocky Linux에는 공식 7 버전이 없으므로, 누군가 ‘Rocky 7’이라고 말해도 실제 /etc/os-release 결과를 확인해야 합니다.

원본과 target의 OS 차이가 크면 오래된 라이브러리나 서비스 실행 방식이 달라질 수 있습니다. 확인 결과가 문서의 가정과 다르면 이관 방법과 호환성 판정을 다시 합니다.

이 부분에서 확인할 것: NO-GO: 실제 OS·kernel·architecture를 모른 채 파일 복원이나 서비스 기동을 진행.

```
cat /etc/os-release
cat /etc/redhat-release
uname -r
uname -m
rpm -E '%{rhel}'
ldd --version
openssl version
```

실측 결과	추가 확인	판정
Rocky Linux 8	DNF module, systemd unit, crypto policy, cgroup, SELinux context, native library ABI	Rocky 8 → OL9.6 분기로 리허설
CentOS/RHEL/Oracle Linux 등 EL7	legacy init/network script, Python 2 의존, 구 OpenSSL/glibc, 서비스 unit 변환, 파일·UID/GID·SELinux label	EL7 → OL9.6 분기로 별도 호환성 결과 기록
그 밖의 배포판/major	패키지·kernel·ABI와 전환 도구 지원 범위 재조사	현 런북 자동 적용 금지, 변경 승인 후 재작성

OS package와 system library는 미리 디렉터리에서 가져오지 않고 OL9.6 저장소/승인 매체로 설치한다.

/application, /mariadb, /logs는 보존 대상이지만 각각의 실행 파일이 OL9.6 library와 실제로 link·기동되는지는 별도 합격 항목이다.

7.2 호스트 호환성

아주 쉽게 설명하면: 7.2 호스트 호환성

기존 프로그램이 새 OS에서 달릴 수 있는지 시험하는 단계입니다.

JAR 파일은 Java가 필요하고, Python 프로그램은 Python과 모듈이 필요하며, native library는 OS의 라이브러리와 CPU 환경의 영향을 받을 수 있습니다. 파일이 있다고 실행되는 것은 아닙니다.

같은 파일을 target에 놓은 뒤 '실행 가능 여부', '로그 오류', '필요한 포트', '설정파일 경로'를 확인합니다. 오류가 나면 소스 코드를 바꾸기보다 먼저 Java 버전·권한·환경변수·라이브러리를 비교합니다.

이 부분에서 확인할 것: 성공 기준: 승인된 JDK/라이브러리에서 실제 JAR과 미들웨어가 오류 없이 기동.

- ☐ Oracle Linux 9.6 release/kernel/architecture, CPU flags, locale, KST/NTP, hostname/IP가 설계와 일치한다.
- ☐ admin 계정의 UID/GID를 확인하고 /application, /mariadb, /logs를 admin:admin으로 정리한다.
- ☐ OpenJDK/JRE 8을 승인된 배포판으로 설치하고 실제 모든 JAR에 대해 java -version과 기동 시험을 한다.
- ☐ Redis/MariaDB 등 native binary마다 동적 라이브러리 누락, OpenSSL ABI, glibc, libaio, zlib을 검사한다.
- ☐ SELinux enforcing 상태에서 서비스별 쓰기 디렉터리와 포트를 허용한다. 문제 해결을 위해 SELinux를 영구 비활성화하지 않는다.
- ☐ systemd unit의 경로, LimitNOFILE/NPROC, vm.max_map_count, transparent huge page, ulimit 값을 source manifest와 대조한다.
- ☐ firewall에는 7000/7001과 cluster bus 17000/17001, ZK 2181/2888/3888, Solr 8983, ES 9200/9300 및 앱 포트를 승인된 내부 대역에만 개방한다.

7.3 파일 복원

아주 쉽게 설명하면: 7.3 파일 복원

프로그램 파일과 설정을 target에 놓는 단계입니다.

JAR·스크립트·설정파일·업로드 파일·systemd unit을 복사하고 checksum과 소유자를 확인합니다. 데이터베이스나 Elasticsearch data 폴더까지 같은 방식으로 덮어쓰라는 뜻은 아닙니다.

서비스 프로그램은 파일 복사로 옮길 수 있지만, 서비스가 서로를 가리키는 IP·hostname·포트는 target 주소에 맞게 별도로 검토해야 합니다.

이 부분에서 확인할 것: 성공 기준: 파일 지문·권한·실행 사용자·설정 주소·로그 경로가 target 기준과 일치.

1. 미러가 보존한 바이너리·설정·service unit을 checksum과 비교한다.

2. JAR은 현재 VM 산출물의 원래 파일명을 그대로 사용한다. 재빌드·개명·Java 코드 수정은 하지 않는다.
3. 각 노드의 CMS 업로드 파일을 별도 staging에 복원하고 DB 참조 무결성 확인 전 합치지 않는다.
4. 로그 디렉터리 /logs/{scheduler,engine,gateway,cms,master,chat-ui,redis,solr,elk,maria}와 /logs/solr/zk의 권한을 확인한다.

기존 결함과 OS 전환 결함 구분

현행 engine에는 fat JAR 내부 js/common.js를 ClassPathResource.getFile()로 접근하는 알려진 기존 결함이 있었다. OL9.6 시험 전 source에서도 같은 증상인지 baseline을 남기고, 기존 결함을 OS 회귀로 오판하지 않는다.

8. MariaDB 미러 기동·Scheduler 데이터 재적재

아주 쉽게 설명하면: 8. MariaDB 미러 기동·Scheduler 데이터 재적재

MariaDB를 원본 미러 기준으로 살리고, Scheduler가 바꾸는 일부 데이터만 정리하는 단계입니다.

MariaDB는 업무의 오래 보존해야 하는 정보가 있을 수 있어 전체를 새로 만들지 않습니다. 미러 DB를 기준으로 유지하고, Scheduler가 생성하거나 변경하는 범위만 source의 최종 INSERT SQL로 target에 맞춥니다.

중요한 순서는 writer와 Scheduler를 멈추고, 데이터가 더 이상 변하지 않는지 확인한 뒤, 승인된 범위만 처리하는 것입니다. target에서 실행하더라도 대상 DB인지 세 번 확인합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: source DB의 현재 데이터, target DB의 table 구조, Scheduler가 건드리는 table 목록, 그리고 승인된 적용 범위입니다. DB는 단순한 파일 창고가 아니라 여러 프로그램이 계속 읽고 쓰는 업무 장부이므로, '파일이 복사됐나'보다 '어떤 행이 같은가'를 확인합니다.

실제 흐름: 먼저 읽기 명령으로 접속·table·행 수·진행 중 작업을 확인합니다. 그다음 writer와 Scheduler를 멈추고 source에서 승인된 data-only INSERT SQL을 만듭니다. target에서는 승인된 범위만 preclear하고 INSERT한 뒤 source와 count·기본키·checksum을 비교합니다. 나머지 업무 원본 table은 건드리지 않는 것이 기본입니다.

1. **읽기:** DB가 살아 있고 구조가 맞는지 봅니다.
2. **고정:** 더 이상 행이 변하지 않도록 writer를 멈춥니다.
3. **적용:** A그룹 전량 또는 B그룹 후보 PK만 처리합니다.
4. **비교:** 다르면 성공으로 보지 않고 멈춥니다.

가장 중요한 이유: MyISAM에서는 실패하면 DB가 자동으로 원래 상태로 돌아온다고 가정하면 안 됩니다. 따라서 지우기 전에 before-image와 원복 INSERT SQL을 먼저 준비합니다.

이 부분에서 확인할 것: 금지: 운영 source에서 전체 삭제, target의 모든 table 무차별 초기화.

DB 처리 원칙

미러된 /mariadb/data와 database/schema/table은 그대로 유지한다. 전체 DB dump-전체 restore-schema 재생성은 하지 않는다. Scheduler가 독점 생성하는 데이터만 source 최종 INSERT SQL로 전량 교체하고, 업무 애플리케이션과 함께 쓰는 상태는 Scheduler 영향 PK만 교정한다.

8.1 미러 DB 무결성 확인

아주 쉽게 설명하면: 8.1 미러 DB 무결성 확인

복사된 DB가 깨지지 않았는지 건강검진하는 단계입니다.

DB에 접속된다는 것과 데이터가 온전하다는 것은 다릅니다. 테이블을 읽을 수 있는지, 행 수가 맞는지, 기본키가 중복되지 않는지, 로그에 오류가 없는지를 따로 확인합니다.

MyISAM은 transaction 처리 특성이 InnoDB와 다르므로 '실패하면 자동 rollback'을 당연하게 생각하면 안 됩니다. 적용 전 before-image와 원복 INSERT SQL을 준비해야 합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: source DB의 현재 데이터, target DB의 table 구조, Scheduler가 건드리는 table 목록, 그리고 승인된 적용 범위입니다. DB는 단순한 파일 창고가 아니라 여러 프로그램이 계속 읽고 쓰는 업무 장부이므로, '파일이 복사됐나'보다 '어떤 행이 같은가'를 확인합니다.

실제 흐름: 먼저 읽기 명령으로 접속·table·행 수·진행 중 작업을 확인합니다. 그다음 writer와 Scheduler를 멈추고 source에서 승인된 data-only INSERT SQL을 만듭니다. target에서는 승인된 범위만 preclear하고 INSERT 한 뒤 source와 count·기본키·checksum을 비교합니다. 나머지 업무 원본 table은 건드리지 않는 것이 기본입니다.

1. **읽기:** DB가 살아 있고 구조가 맞는지 봅니다.
2. **고정:** 더 이상 행이 변하지 않도록 writer를 멈춥니다.
3. **적용:** A그룹 전량 또는 B그룹 후보 PK만 처리합니다.
4. **비교:** 다르면 성공으로 보지 않고 멈춥니다.

가장 중요한 이유: MyISAM에서는 실패하면 DB가 자동으로 원래 상태로 돌아온다고 가정하면 안 됩니다. 따라서 지우기 전에 before-image와 원복 INSERT SQL을 먼저 준비합니다.

이 부분에서 확인할 것: 성공 기준: table/checksum/count/auto_increment와 오류 로그 확인.

1. 정확한 MariaDB 버전과 plugin, lower_case_table_names, charset/collation, time_zone, sql_mode를 source manifest와 맞춘다.
2. 최초 기동 전 /mariadb/data를 변경 불가능한 VM snapshot 또는 `/${HOLD_ROOT}/mariadb/<node>/data.hotmirror.raw`로 보관하고 working copy에서만 검사한다.
3. 운영망·원본 DB와 격리한 상태로 미러 DB를 기동한다. error log, system table과 모든 업무 MyISAM table을 검사한다.
4. crashed/corrupt table이 있으면 raw hold를 건드리지 말고 복제 working copy에서 repair 가능성을 검토한다. 복구 불가 table만 DB 솔루션 담당 절차로 행 단위 복구한다.
5. 업무 사용자·GRANT는 확인하되 system DB를 다른 버전으로 덮어쓰지 않는다.

```

mariadb-check \
  --defaults-extra-file=<TARGET_DB_OPTION_FILE> \
  --host="${TGT_DB_HOST}" --port="${TGT_DB_PORT}" \
  --all-databases --check

```

8.2 Scheduler 간섭 데이터 경계

아주 쉽게 설명하면: 8.2 Scheduler 간섭 데이터 경계

어떤 DB 행을 다시 넣고 어떤 행은 건드리지 않을지 선을 긋는 단계입니다.

Scheduler는 통계·배치 기록을 만들지만, CMS나 Master도 공용 상태를 바꿀 수 있습니다. 모든 table을 Scheduler 데이터라고 생각하면 업무 원본을 지울 수 있습니다.

A그룹은 승인된 Scheduler 전용 데이터라면 전량 교체하고, B그룹은 영향받은 후보 PK만 고칩니다. 그 외 업무 원본 table은 미리 상태를 유지합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: Scheduler 실행 node, 실행 주기, job 이름, 대상 DB·Solr·Redis·Elasticsearch, 진행 중 job 목록입니다. Scheduler는 사람이 버튼을 누르지 않아도 시간에 맞춰 일하는 자동 직원입니다.

실제 흐름: source와 target의 자동 직원이 동시에 출근하지 않게 합니다. 진행 중 job이 끝났는지 확인하고 source Scheduler를 중지합니다. 리허설에서는 승인된 한 대만 제한 기동하고 배포·ES 로그 삭제 job은 별도 승인합니다.

1. 실제 실행 node를 찾습니다.
2. 진행 중·예약 job을 확인합니다.
3. source Scheduler를 멈춥니다.
4. target 한 대에서 필요한 job만 시험합니다.
5. DB·검색·cache 변화를 비교합니다.

큰 위험: 101·102·103이 동시에 실행되면 같은 통계나 배포가 중복될 수 있습니다. 그래서 Scheduler는 전체 검증의 마지막에 한 대만 컵니다.

이 부분에서 확인할 것: 성공 기준: 대상 table 목록과 제외 table 목록에 담당자 승인.

처리	대상	적용 단위
전량 교체 A	master: tbl_batch_log 각 tenant: tbl_api_statistics, tbl_dialog_statistics, tbl_knowledge_statistics, tbl_satisfaction_statistics, tbl_scenario_statistics, tbl_scenario_node_statistics, tbl_slot_statistics, tbl_top10_dialog_statistics, tbl_user_statistics	target 해당 table의 데이터만 DELETE한 뒤 source B1 INSERT SQL 전량 적재
선택 교정 B	tenant: tbl_learn_recommend, tbl_deploy, tbl_bot master: tbl_manage_deploy	source와 target 후보 PK 합 집합만 원복 백업 후 교정. tbl_bot은 active/standby deploy-learn 상태 컬럼만 UPDATE
기본 제외	tbl_learning_info, tbl_manage_monitoring_response, tbl_job_instance, answer-link/additional/payment 계열	현재 소스에서 Scheduler 쓰 기가 확인되지 않거나 job이 비활성·미확인이다. 실제 배포 JAR 분석으로 활성 쓰기가 입 증된 경우에만 승인 후 범위 추가

리허설 데이터 제한

이 방식은 target 시험 중 Scheduler 외 업무 DML을 만들지 않는다는 전제다. CMS에서 실제 bot-deploy-learn 업무행을 생성했다면 해당 PK를 별도 “시험 데이터 폐기 목록”으로 관리해야 하며, Scheduler 간섭 데이터와 섞어 자동 삭제하지 않는다.

8.3 Source B0/B1 data-only INSERT SQL

아주 쉽게 설명하면: 8.3 Source B0/B1 data-only INSERT SQL

source의 데이터를 '구조 없이 행만 넣는 SQL'로 준비하는 단계입니다.

data-only SQL은 CREATE TABLE 같은 구조 생성문보다 INSERT 문을 중심으로 만든 파일입니다. target의 구조는 먼저 준비되어 있다고 보고, 필요한 행을 넣는 데 사용합니다.

B0는 연습용, B1은 최종 전환 직전의 마지막 변경분입니다. 파일 이름과 생성 시각을 기록하고 SHA-256을 계산해 전송 중 변경되지 않았는지 확인합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: 원본의 현재 상태, 백업 저장 위치, 생성 시각, checksum, 복원 방법입니다. 백업은 파일 한 개를 복사하는 행동이 아니라 실패했을 때 돌아갈 지점을 만드는 일입니다.

B0와 B1: B0는 리허설 전 복원 연습용이고, B1은 최종 전환 직전 writer와 Scheduler를 멈춘 뒤 마지막 변경분을 고정하는 용도입니다. 이름이 비슷해도 시점과 목적이 다릅니다.

1. 백업 대상을 정합니다.
2. 파일·시각·크기를 기록합니다.
3. SHA-256으로 변조 여부를 봅니다.
4. target에서 실제 복원합니다.
5. 성공·실패와 승인을 남깁니다.

초보자 원칙: '백업 성공' 메시지만 믿지 말고 목록·checksum·복원 결과까지 확인합니다.

이 부분에서 확인할 것: 성공 기준: source 시각·범위·행 수·checksum·원복 SQL 기록.

1. B0에는 미리 시작시각보다 충분히 앞선 B0_SAFE, 당시 예약대기 deploy ID, A/B table count·checksum·AUTO_INCREMENT를 기록한다. 기본 안전여유는 최소 2시간이며 미리 시작시각이 불확실하면 전날 00:00까지 넓혀 잡는다.
2. B1에는 source Scheduler를 먼저 멈추고 진행 job 0건을 확인한다. B 공용 table 추출 동안 CMS/Master writer도 멈춰 같은 시점을 보장한다.
3. master DB의 tbl_batch_log와 모든 활성 tenant DB의 9개 통계 table을 DB별 A그룹 data-only SQL로 만든다. 테스트베드처럼 같은 schema일 때만 합칠 수 있다. 파일은 SHA-256과 dump 종료코드가 있어야 한다.

```

mariadb-dump \
  --defaults-extra-file=<SOURCE_DB_OPTION_FILE> \
  --host="${SRC_DB_HOST}" --port="${SRC_DB_PORT}" \
  --no-create-info --complete-insert --skip-extended-insert \
  --skip-triggers --quick --lock-all-tables --hex-blob \
  --default-character-set=utf8mb4 --order-by-primary \
  --skip-add-locks --skip-disable-keys --skip-dump-date \
  "${TENANT_DB_NAME}" \
  tbl_api_statistics tbl_dialog_statistics \
  tbl_knowledge_statistics tbl_satisfaction_statistics \
  tbl_scenario_statistics tbl_scenario_node_statistics \
  tbl_slot_statistics tbl_top10_dialog_statistics tbl_user_statistics \
  > "${BACKUP_ROOT}/mariadb/${TENANT_DB_NAME}.B1.statistics.insert.sql"

mariadb-dump \
  --defaults-extra-file=<SOURCE_DB_OPTION_FILE> \
  --host="${SRC_DB_HOST}" --port="${SRC_DB_PORT}" \
  --no-create-info --complete-insert --skip-extended-insert \
  --skip-triggers --quick --lock-all-tables --hex-blob \
  --default-character-set=utf8mb4 --order-by-primary \
  --skip-add-locks --skip-disable-keys --skip-dump-date \
  "${MASTER_DB_NAME}" tbl_batch_log \
  > "${BACKUP_ROOT}/mariadb/${MASTER_DB_NAME}.B1.batch-log.insert.sql"

sha256sum "${BACKUP_ROOT}/mariadb/"*.B1.*.insert.sql

```

B그룹 후보는 source와 target 양쪽에서 $B0_SAFE \leq created_at/modified_at/finished_at/request_date < B1$ 인 PK, B0 예약대기 deploy ID와 그 bot/deploy 연관 PK를 뽑아 합집합으로 만든다. source 목록만 쓰면 target Scheduler가 만든 행을 놓치므로 금지한다. 검증된 숫자 PK 목록을 `--where="id IN (...)"`으로 추출하며, source에서 삭제된 후보는 INSERT 파일에 없다는 점을 manifest에 표시한다.

```
-- source와 target 양쪽에서 실행한 뒤 PK 합집합을 승인한다.
SELECT id FROM tbl_learn_recommend
WHERE (created_at >= :B0_SAFE AND created_at < :B1)
      OR (modified_at >= :B0_SAFE AND modified_at < :B1)
      OR (request_date >= :B0_SAFE AND request_date < :B1);

SELECT id, bot_id FROM tbl_deploy
WHERE deploy_type = 'SCHEDULE'
      AND (id IN (:B0_PENDING_DEPLOY_IDS)
           OR (created_at >= :B0_SAFE AND created_at < :B1)
           OR (modified_at >= :B0_SAFE AND modified_at < :B1)
           OR (finished_at >= :B0_SAFE AND finished_at < :B1)
           OR (deploy_time >= :B0_SAFE AND deploy_time < :B1));

SELECT id, deploy_id FROM tbl_manage_deploy
WHERE deploy_id IN (:DEPLOY_ID_UNION)
      OR (created_at >= :B0_SAFE AND created_at < :B1)
      OR (modified_at >= :B0_SAFE AND modified_at < :B1);
```

8.4 Target 데이터 비우기·재적재

아주 쉽게 설명하면: 8.4 Target 데이터 비우기·재적재

target에서 승인된 범위의 오래된 행을 치우고 **source** 행을 넣는 단계입니다.

‘데이터를 비운다’는 말은 DB 전체를 지운다는 뜻이 아닙니다. 승인된 A그룹 table 또는 특정 범위만 대상으로 합니다. target 식별을 확인한 뒤 preclear 백업과 원복 SQL이 있어야 합니다.

MyISAM에서는 중간에 실패해도 자동으로 되돌아가지 않을 수 있으므로, 적용 전후 행 수와 checksum을 비교합니다. 실패하면 미리 만든 rollback INSERT SQL로 되돌립니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: source VM의 CPU·RAM·디스크·NIC·OS·서비스와 target host의 저장공간·네트워크·자원입니다. VM은 가상 CPU·메모리·디스크·네트워크 카드를 가진 컴퓨터입니다.

실제 흐름: 가상화 담당자가 VM을 복사하거나 export/import합니다. 복사본을 처음 켤 때는 원본과 다른 임시 IP·hostname을 사용합니다. 부팅 후 OS·mount·/data·자동기동 서비스·Scheduler 상태를 확인합니다.

1. source 상태와 백업을 고정합니다.
2. target 저장소와 network를 확인합니다.
3. VM을 복사·인수합니다.
4. 임시 주소로 격리 부팅합니다.
5. 원본과 충돌이 없는지 확인합니다.

비유: VM 복사는 집 전체를 새 동네에 복사하는 일입니다. 주소판과 전화번호가 예전 집과 같으면 우편과 전화가 섞이므로, 새 주소를 붙이고 손님을 받기 전까지 격리합니다.

이 부분에서 확인할 것: 변경 전 필수: target 확인, writer 정지, preclear 백업, 승인.

1. target Scheduler와 모든 application writer를 중지하고 대상 DB의 hostname/IP/schema를 다시 확인한다.
2. master의 batch log와 각 tenant의 9개 통계 table에 있는 target 현 데이터를 같은 data-only 형식의 DB별 target-preclear.rollback.insert.sql로 추출하고 SHA-256-count-checksum을 기록한다.
3. B1 INSERT SQL을 격리 staging DB의 동일 DDL table에 먼저 적재해 문법, row count, PK/unique 중복, CHECK TABLE과 source checksum을 검증한다.
4. 각 대상 DB의 한 client session에서 그 DB의 A그룹 table을 WRITE lock하고 명시적 DELETE FROM으로 데이터만 비운 뒤 해당 B1 SQL을 적재한다. 실패하면 writer를 계속 정지하고 그 DB의 rollback INSERT SQL로 원복한다.
5. 각 table의 AUTO_INCREMENT를 source B1 manifest 값 이상으로 맞춘 뒤 count-checksum을 즉시 비교한다.

아래 묶음 SQL은 master와 tenant가 같은 meritx_easycms인 현 테스트베드용 예시다. 운영에서 DB가 분리되어 있으면 master에는 tbl_batch_log만, 각 tenant에는 9개 통계 table만 따로 lock-delete-import한다.

LOCK TABLES

```
tbl_batch_log WRITE, tbl_api_statistics WRITE,
tbl_dialog_statistics WRITE, tbl_knowledge_statistics WRITE,
tbl_satisfaction_statistics WRITE, tbl_scenario_statistics WRITE,
tbl_scenario_node_statistics WRITE, tbl_slot_statistics WRITE,
tbl_top10_dialog_statistics WRITE, tbl_user_statistics WRITE;
```

```
DELETE FROM tbl_batch_log;
DELETE FROM tbl_api_statistics;
DELETE FROM tbl_dialog_statistics;
DELETE FROM tbl_knowledge_statistics;
DELETE FROM tbl_satisfaction_statistics;
DELETE FROM tbl_scenario_statistics;
DELETE FROM tbl_scenario_node_statistics;
DELETE FROM tbl_slot_statistics;
DELETE FROM tbl_top10_dialog_statistics;
DELETE FROM tbl_user_statistics;
```

```
SOURCE /secure/<validated-B1-file-for-this-DB>.insert.sql;
UNLOCK TABLES;
```

위 SQL은 대상 DB별 실행 템플릿이다. SOURCE가 같은 client session 안에서 성공했는지 확인하며, MyISAM에는 transaction rollback이 없으므로 중간 실패를 ROLLBACK으로 복구할 수 없다.

8.5 공용 상태 B그룹 선택 교정

아주 쉽게 설명하면: 8.5 공용 상태 B그룹 선택 교정

업무 원본을 보존하면서 정말 영향을 받은 행만 고치는 단계입니다.

B그룹은 Scheduler뿐 아니라 CMS·Master·사용자 요청도 함께 만질 수 있는 공용 상태입니다. 그래서 전체 삭제 후 재적재하면 안 되고, source와 target의 후보 PK를 비교해 필요한 행만 교정합니다.

후보 PK는 '이 행이 달라졌을 가능성이 있다'는 목록이지 무조건 지워도 된다는 뜻이 아닙니다. 각 PK의 before/after 값을 기록하고 DB 담당자의 승인을 받습니다.

이 부분에서 확인할 것: NO-GO: 후보 범위가 정해지지 않았거나 before-image가 없음.

1. 후보 PK 합집합의 target 원본 행을 table별 rollback SQL 또는 backup table에 먼저 저장한다.
2. tbl_manage_deploy, tbl_learn_recommend, tbl_deploy의 후보 PK만 target에서 삭제하고 source B1에 실제 존재하는 후보 행만 INSERT한다. source에 없는 후보는 삭제 상태로 남긴다.
3. tbl_bot은 행 전체를 교체하지 않는다. 후보 bot의 active_deploy_id, active_learn_id, standby_deploy_id, standby_learn_id와 대응 수정정보만 source staging 값으로 UPDATE한다.

4. 각 명령의 예상/실제 affected rows를 비교하고 다르면 즉시 중단한다. MyISAM이므로 실패 시 준비된 target before-image로 역순 원복한다.

8.6 검증·기동 제한

아주 쉽게 설명하면: 8.6 검증·기동 제한

DB를 확인한 뒤 애플리케이션을 아주 제한적으로 연결하는 단계입니다.

먼저 DB 자체가 맞는지 확인하고, 그 다음 Master-Engine 같은 프로그램을 붙입니다. Scheduler는 여러 VM에서 동시에 실행되지 않도록 마지막까지 꺼 둡니다.

리허설에서는 101 한 대만 승인된 job을 제한적으로 실행하고 결과를 비교합니다. target에서 만든 시험 이력이 최종 운영 DB에 섞이지 않도록 처리합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: DB-Redis-검색 서비스 health 결과, JAR, 환경변수, 설정파일, 실행 사용자, 서비스 의존관계입니다. 애플리케이션은 다른 서비스에 전화를 거는 프로그램입니다.

실제 흐름: 데이터 서비스가 준비된 뒤 Master-Engine-Gateway-CMS-Chat UI를 연결합니다. 각 서비스마다 프로세스·port-API-로그를 확인하고 source 주소를 보고 있지 않은지 검색합니다.

1. 파일과 설정 주소를 확인합니다.
2. 의존 서비스가 정상인지 봅니다.
3. 한 서비스씩 기동합니다.
4. API와 업무 흐름을 검증합니다.

기억할 점: systemctl active는 1차 신호일 뿐입니다. 실제 성공은 API·연결·로그인·검색·시뮬레이터까지 확인해야 합니다.

이 부분에서 확인할 것: 성공 기준: DB 정합성 승인, Scheduler 중복 없음, 주요 읽기·검색 업무 통과.

- ☐ A그룹은 source/target의 count, MIN/MAX PK, AUTO_INCREMENT, checksum, unique 중복, CHECK TABLE이 일치한다.
- ☐ B그룹은 후보 PK별 source/target 행과 deploy 결과·완료시각, manage deploy, learn recommend, bot의 네 상태 컬럼이 일치한다.
- ☐ A/B 이외 table의 전후 count-checksum이 동일하다. 차이가 있으면 “Scheduler 간섭 데이터만 변경” 조건 실패다.
- ☐ tbl_deploy.bot_id → tbl_bot.id, learning 정보, bot active/standby deploy-learn, tenant tbl_manage_deploy.deploy_id의 논리 orphan이 없다.
- ☐ 기한이 지난 STANDBY/SCHEDULE 예약배포를 전수 확인하고 업무 승인 전 Scheduler를 기동하지 않는다.

DB 정합화 전 애플리케이션 선기동 금지

master의 초기화 코드와 CMS의 schema update, Scheduler의 batch 초기화가 DB를 변경할 수 있다. 최초 검증에서는 승인된 외부 override로 DDL 자동 변경과 Quartz 자동 시작을 막고, 선택 재적재·검증이 끝난 뒤에만 정상 설정으로 전환한다.

9. Redis 5.0.8 재클러스터링·cache 재배포

아주 쉽게 설명하면: 9. Redis 5.0.8 재클러스터링·cache 재배포

빠른 임시 참고인 Redis를 target에서 정상화하는 단계입니다.

Redis에는 세션·cache·큐처럼 다시 만들 수 있는 값이 있을 수 있지만, 실제 업무에서 무엇을 저장하는지는 설정을 확인해야 합니다. 여러 Redis가 하나의 cluster라면 node와 slot 관계도 같이 맞아야 합니다.

기존 cluster가 target에서 안전하게 재결합되면 그 경로를 사용하고, 상태가 꼬이면 새 빈 cluster를 만들고 애플리케이션의 배포 기능으로 cache를 재생성합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: Redis 노드의 주소·port·master/replica 역할·node ID·slot 상태와 AOF/RDB 파일 정보입니다. Redis가 cache만 저장하는지, 로그인 세션·queue·업무 상태도 저장하는지는 애플리케이션 설정으로 확인합니다.

실제 흐름: PONG은 한 노드가 대답했다는 뜻이고, cluster_state:ok와 모든 slot 정상은 집단 전체가 일할 준비가 됐다는 뜻입니다. 기존 cluster 재결합이 안전하면 유지하고, 주소·node ID·slot이 꼬이면 빈 3 master/3 replica를 구성한 뒤 cache를 다시 만듭니다.

1. 노드와 역할을 기록합니다.
2. source와 target이 동시에 같은 cluster가 되지 않게 합니다.
3. 재결합 또는 새 cluster 중 하나를 선택합니다.
4. cache 재생성 뒤 TTL·대표 기능을 확인합니다.

주의: 'Redis니까 지워도 된다'가 아닙니다. cache라면 다시 만들 수 있지만 세션·queue라면 초기화가 곧 업무 영향일 수 있으므로 담당자 승인이 필요합니다.

이 부분에서 확인할 것: 금지: source와 target을 같은 cluster에 무검증으로 섞기, source에서 FLUSHALL.

Redis 데이터 이관 없음

Redis는 재생성 가능한 cache이므로 source에서 B0/B1 RDB를 별도로 옮겨 import하지 않는다. 미래본의 cluster state·AOF·RDB가 정상이라면 그대로 재결합할 수 있고, 그렇지 않으면 빈 cluster를 만든 뒤 시스템 배포 기능으로 cache를 재생성한다.

9.1 파일과 금지 조합

아주 쉽게 설명하면: 9.1 파일과 금지 조합

Redis 파일을 어떤 식으로 다룰지 정하는 안전장치입니다.

RDB는 특정 시점의 메모리 저장 파일이고 AOF는 명령 기록에 가까운 파일입니다. cluster state와 설정이 서로 다른 시점이면 파일을 함께 넣었을 때 노드가 이상해질 수 있습니다.

따라서 파일을 발견했다고 target에 바로 덮어쓰지 않고, 경로·생성 시각·checksum을 보관한 후 재결합 또는 새 cluster 판단을 합니다.

이 부분에서 확인할 것: 금지: 서로 다른 시점의 RDB/AOF/cluster state를 섞어 무검증 기동.

- ☐ 정적 설정은 /application/redis/conf/7000.conf, 7001.conf다.
- ☐ Redis가 자동 관리하는 cluster state는 각 data directory의 /application/redis/db/7000/7000.conf, /application/redis/db/7001/7001.conf다. 같은 파일명이어도 정적 설정과 혼동하거나 직접 편집하지 않는다.
- ☐ 각 data directory의 appendonly.aof, dump.rdb와 자동 cluster state를 한 세트로 취급한다. 현재 appendonly yes이므로 재기동 시 AOF가 RDB보다 우선될 수 있다.
- ☐ **cluster state만 지우고 AOF/RDB를 남기는 방식은 금지한다.** 새 node ID의 non-empty master가 되어 표준 --cluster create, reset-replica 전환이 막힐 수 있다.

9.2 우선 경로 — 미리 cluster 그대로 재결합

아주 쉽게 설명하면: 9.2 우선 경로 — 미리 cluster 그대로 재결합

기존 Redis 집단을 새 장소에서 다시 서로 찾게 하는 경로입니다.

재결합은 Redis 노드들이 서로의 node ID와 주소를 알고 cluster의 16,384 slot을 정상적으로 나눠 맡도록 만드는 것입니다. 단순히 Redis 프로세스를 켜는 것보다 확인해야 할 것이 많습니다.

cluster info가 ok인지, 모든 slot이 배정됐는지, master와 replica가 예상대로 보이는지, 애플리케이션이 cache를 읽는지를 확인합니다.

이 부분에서 확인할 것: 성공 기준: cluster_state:ok, cluster_slots_ok:16384, replica 연결 정상.

1. 동일 node ID를 가진 source와 target이 같은 네트워크에 동시에 노출되지 않도록 source cluster를 완전히 격리한다.
2. target의 /etc/hosts에서 redis1/2/3이 target 주소를 가리키는지, 7000/7001 및 cluster bus 17000/17001이 상호 통신되는지 확인한다.
3. 여섯 노드의 정적 설정, 자동 state, AOF, RDB를 변경 전 hold에 복사하고 checksum을 남긴다.

4. 7000 세 master를 먼저 기동하고 7001 세 replica를 기동한다. 기존 state에 hostname이 기록되어 있더라도 실제 announce 주소와 해석 결과를 확인한다.
5. 필요한 최초 접점에 한해 새 주소로 CLUSTER MEET를 사용할 수 있으나 자동 state 파일은 편집하지 않는다.
6. 아래 합격값을 모두 만족하면 기존 RDB/AOF를 그대로 둔 채 다음 단계로 간다. 하나라도 실패하면 수동 slot 억지 복구를 중단하고 9.3의 빈 cluster 경로로 전환한다.

```
cluster_state:ok
cluster_slots_assigned:16384
cluster_slots_ok:16384
cluster_known_nodes:6
cluster_size:3
master 3 / replica 3
fail, fail?, handshake, noaddr, migrating, importing = 0
```

9.3 대체 경로 — 빈 3 master / 3 replica 재구성

아주 쉽게 설명하면: 9.3 대체 경로 — 빈 3 master / 3 replica 재구성

기존 Redis 상태를 억지로 고치지 않고 새로 구성하는 경로입니다.

빈 cluster는 cache가 없는 깨끗한 Redis 집단입니다. master 3대와 replica 3대를 만들고 slot을 나눈 뒤, 애플리케이션이 필요한 cache를 다시 채웁니다.

이 방식은 cache가 정말 재생성 가능한 경우에만 안전합니다. 세션·큐·업무상태가 Redis에 있다면 데이터 손실 가능성을 담당자와 먼저 판단해야 합니다.

이 부분에서 확인할 것: GO 조건: cache 재생성 방법과 데이터 손실 허용 범위 승인.

1. target 7000/7001 여섯 프로세스를 중지한다.
2. 각 인스턴스의 자동 cluster state, appendonly.aof, dump.rdb를 \${HOLD_ROOT}/redis/<node>/<port>로 이동한다. 정적 /application/redis/conf/*.conf는 유지한다.
3. 빈 working directory와 admin:admin 권한을 만들고 여섯 인스턴스를 기동한다. 전부 DBSIZE=0, cluster_known_nodes=1이고 node ID 여섯 개가 서로 다른지 확인한다.
4. 7000 세 개로 master cluster를 만든 후, 7001을 다른 물리 VM master의 replica로 교차 배치한다.

```
/application/redis/bin/redis-cli --cluster create \
redis1:7000 redis2:7000 redis3:7000 \
--cluster-replicas 0 --cluster-yes
```

```
# 실제 7000 master node ID를 조회한 뒤 --cluster add-node로 교차 배치
redis1:7001 → redis2:7000 replica
redis2:7001 → redis3:7000 replica
redis3:7001 → redis1:7000 replica
```

현장 Redis 5.0.8의 `redis-cli --cluster help`로 `add-node --cluster-slave --cluster-master-id` 옵션을 먼저 확인한다. 재구성 후에도 9.2의 cluster 합격값을 모두 만족해야 한다.

9.4 시스템 배포 기능으로 cache 재생성

아주 쉽게 설명하면: 9.4 시스템 배포 기능으로 cache 재생성

빈 Redis에 애플리케이션이 필요한 값을 다시 만들어 넣는 단계입니다.

Redis를 복원했다는 말은 모든 예전 cache 값을 그대로 가져왔다는 뜻이 아니라, 업무 시스템이 다시 실행되면서 필요한 값을 재생성했다는 뜻일 수 있습니다.

재생성 뒤 대표 화면·로그인·검색·챗봇 등 실제 사용 흐름에서 cache miss가 오류로 이어지지 않는지 확인합니다.

이 부분에서 확인할 것: 성공 기준: 대표 cache 조회, TTL, 애플리케이션 로그, 시뮬레이터 결과 정상.

1. Scheduler는 계속 OFF로 두고 DB·Solr·Elasticsearch 최종 복원 및 관련 애플리케이션의 내부 health 검증을 먼저 끝낸다.
2. 승인된 시스템 배포 기능으로 전체 bot/cache를 한 번만 재배포한다. 실행 ID, 대상 bot, 시작·종료 시각과 성공/실패를 기록한다.
3. 재배포 기능이 stale key 삭제 없이 upsert만 수행한다면 기존 persistence 유지 경로를 합격 처리하지 않는다. 9.3의 빈 cluster에서 다시 배포한다.
4. 대표 cache key의 type/value/TTL, cluster-aware read/write, 각 master의 key 분산, replica 동기화와 애플리케이션 조회를 확인한다.

10. ZooKeeper 3.6.1 / Solr 7.6.0 재구성·복원

아주 쉽게 설명하면: 10. ZooKeeper 3.6.1 / Solr 7.6.0 재구성·복원

검색 시스템의 조정자와 검색 문서를 새 환경에 복원하는 단계입니다.

ZooKeeper는 SolrCloud의 반장처럼 collection-replica-leader 정보를 조정합니다. Solr는 검색 API와 문서를 제공합니다. 둘이 연결되지 않으면 Solr 프로세스가 떠 있어도 검색이 정상이라고 할 수 없습니다.

기존 data 디렉터리를 단순 복사하는 대신 clean ensemble, configset, collection backup/restore 순서로 구성합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: ZooKeeper ensemble 주소와 myid, Solr node 주소, configset, collection 목록, shard-replica-leader-alias 정보입니다. ZooKeeper는 SolrCloud의 조정자이고 Solr는 실제 검색 문서를 제공하므로 둘을 한 세트로 봅니다.

실제 흐름: 새 ZooKeeper가 quorum을 만든 뒤 검색 규칙인 configset을 올립니다. 그다음 source의 모든 collection을 하나씩 BACKUP/RESTORE합니다. 복원 후에는 이름만 보지 않고 leader-replica-alias-문서 수-대표 한국어 검색을 확인합니다.

1. 새 ZK ensemble을 준비합니다.
2. Solr가 새 ZK를 보게 합니다.
3. configset과 collection을 복원합니다.
4. shard-replica-leader-alias-검색을 검사합니다.

자주 하는 착각: Solr 8983 port가 열려 있어도 collection이나 leader가 없으면 검색 시스템은 완성되지 않은 것입니다. ZK의 ruok 응답도 프로세스 생존 확인일 뿐 quorum 전체 정상 판정은 아닙니다.

이 부분에서 확인할 것: 성공 기준: 모든 collection active, shard/replica/leader 정상, alias와 문서 수 일치.

Collection 이관은 필수 게이트

미러된 Solr index directory나 ZooKeeper transaction log를 최종 데이터로 인정하지 않는다. TC_CALLBOT_A, TC_CALLBOT_B, TC_CALLBOT_LOG, TC_CALLBOT_TEST를 포함해 source에 존재하는 모든 collection이 최종 BACKUP/RESTORE로 이관되어야 하며 하나라도 누락·실패하면 cutover는 NO-GO다.

10.1 ZooKeeper clean ensemble

아주 쉽게 설명하면: 10.1 ZooKeeper clean ensemble

오래된 회의 기록을 버리고 새 회의체를 만드는 단계입니다.

ZooKeeper ensemble은 여러 ZooKeeper 서버가 서로 합의하는 묶음입니다. 이전 서버의 임시 세션이나 오래된 transaction log까지 그대로 가져오면 새 환경에서 잘못된 서버를 기억할 수 있습니다.

새 ensemble을 만들고 각 서버의 myid와 server 목록을 맞춥니다. quorum이 만들어진 뒤 Solr가 새 ZooKeeper 주소를 바라보게 합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: ZooKeeper ensemble 주소와 myid, Solr node 주소, configset, collection 목록, shard-replica-leader-alias 정보입니다. ZooKeeper는 SolrCloud의 조정자이고 Solr는 실제 검색 문서를 제공하므로 둘을 한 세트로 봅니다.

실제 흐름: 새 ZooKeeper가 quorum을 만든 뒤 검색 규칙인 configset을 올립니다. 그다음 source의 모든 collection을 하나씩 BACKUP/RESTORE합니다. 복원 후에는 이름만 보지 않고 leader-replica-alias-문서 수·대표 한국어 검색을 확인합니다.

1. 새 ZK ensemble을 준비합니다.
2. Solr가 새 ZK를 보게 합니다.
3. configset과 collection을 복원합니다.
4. shard-replica-leader-alias-검색을 검사합니다.

자주 하는 착각: Solr 8983 port가 열려 있어도 collection이나 leader가 없으면 검색 시스템은 완성되지 않은 것입니다. ZK의 ruok 응답도 프로세스 생존 확인일 뿐 quorum 전체 정상 판정은 아닙니다.

이 부분에서 확인할 것: 금지: source와 target ZooKeeper를 같은 ensemble으로 무검증 연결.

1. Solr와 ZooKeeper를 모두 중지하고 실제 dataDir, dataLogDir를 hold 영역으로 이동한다.
2. 새 빈 디렉터리를 만들고 세 노드에 같은 server 목록을 설정한다: zk1:2888:3888, zk2:2888:3888, zk3:2888:3888.
3. 각 dataDir에 고유 myid 1/2/3을 두고 신규 빈 ensemble임을 나타내는 initialize marker를 생성한다.
4. 세 노드를 기동해 leader 1, follower 2, client 2181, quorum/election 2888/3888, 동일 membership을 확인한다.

미러본의 version-2, old epoch, /live_nodes와 ephemeral session은 복원하지 않는다.

10.2 configset과 collection 복원

아주 쉽게 설명하면: 10.2 configset과 collection 복원

검색 규칙과 검색 문서 묶음을 되살리는 단계입니다.

configset은 '어떤 필드로 검색하고 분석할지' 정한 설계도이고, collection은 실제 문서 저장소입니다. 먼저 설계를 올리고, 그 다음 모든 collection을 복원합니다.

collection 이름 하나라도 빠지면 일부 기능만 정상처럼 보일 수 있습니다. alias, shard, replica, leader, 문서 수, 대표 한국어 검색까지 확인해야 합니다.

이 부분에서 확인할 것: NO-GO: source collection 목록과 target collection 목록이 다름.

1. 필요한 security.json을 승인 후 먼저 적용하고 chat-base-config를 원래 이름으로 zk upconfig한다.
2. 세 노드에서 SOLR_HOST=solr1/2/3, ZK_HOST=zookeeper1:2181,zookeeper2:2181,zookeeper3:2181, ktnori JAR 위치/checksum, /logs/solr 권한을 확인한다.
3. Solr 세 노드를 켜고 live node가 정확히 3개인지 확인한다.
4. source manifest와 target collection 목록을 대조해 복원 전 동일 이름 collection이 없음을 확인한다. 리허설은 B0, 최종 전환은 B1 BACKUP을 collection별 RESTORE하고 모든 비동기 요청의 completed를 확인한다.
5. collection이 active가 된 뒤 alias TC_CALLBOT과 승인된 cluster property를 API로 재구성한다. old znode 파일을 원시 복사하지 않는다.

```
curl -fsSG "${TGT_SOLR_URL}/solr/admin/collections" \
  --data-urlencode "action=RESTORE" \
  --data-urlencode "name=${BACKUP_NAME}" \
  --data-urlencode "location=${SOLR_BACKUP_LOCATION}" \
  --data-urlencode "collection=${COLLECTION}" \
  --data-urlencode "async=${REQUEST_ID}" --data-urlencode "wt=json"
```

검증 기준: source collection 이름 집합과 target 이름 집합 완전 일치, 모든 shard/replica active, leader 존재, collection별 numFound 일치, replica 물리 분산, 특히 TC_CALLBOT_A가 solr3 한 대에 물리지 않음, alias/configset 일치, 대표 ID 조회, 한국어 분석 API와 ktnori 색인/검색, 1건 생성·조회·삭제, Solr 한 노드와 ZK follower 한 노드의 개별 재기동 후 정상 복구.

Solr collection 이관 완료표

☐ TC_CALLBOT_A ☐ TC_CALLBOT_B ☐ TC_CALLBOT_LOG ☐ TC_CALLBOT_TEST ☐ 추가 발견
collection 전부 ☐ alias TC_CALLBOT ☐ chat-base-config/ktnori

11. Elasticsearch 7.8.0 / Kibana 7.8.0 재클러스터링 ·Snapshot 복원

아주 쉽게 설명하면: 11. Elasticsearch 7.8.0 / Kibana 7.8.0 재클러스터링 ·Snapshot 복원

검색·분석 시스템을 새 cluster로 만들고 Snapshot을 읽어오는 단계입니다.

Elasticsearch는 node와 shard가 있는 검색 저장소입니다. node data 폴더를 통째로 복사하면 cluster UUID·shard 상태·버전 문제를 만들 수 있으므로 새 빈 cluster를 만들고 공식 Snapshot을 복원합니다.

Kibana 화면도 index와 연결 설정이 필요합니다. Elasticsearch 데이터와 정책 객체를 각각 확인하고, target에서 source로 쓰는 연결이 없는지 검사합니다.

조금 더 자세하: 실제 작업의 흐름

들어오는 자료: Elasticsearch node·cluster 정보, index·shard·alias 목록, Snapshot repository, template·pipeline·ILM·SLM 정책입니다. Elasticsearch는 index를 여러 shard로 나누어 저장하므로 data 폴더를 일반 파일처럼 복사하지 않습니다.

실제 흐름: target에서 새 빈 cluster를 준비하고 repository를 등록한 뒤 성공한 Snapshot을 restore합니다. 복원 후 cluster health·node·index·shard를 보고, 정책 객체는 별도로 비교합니다. green은 shard 배치가 정상이라는 뜻이지 업무 검색까지 자동 보장한다는 뜻은 아닙니다.

1. 새 cluster와 repository를 준비합니다.
2. 필요한 snapshot과 index를 복원합니다.
3. template·pipeline·ILM·SLM을 따로 확인합니다.
4. 대표 검색·정렬·aggregation·재기동을 시험합니다.

raw data 폴더를 피하는 이유: 그 안에는 cluster UUID와 shard 상태 같은 내부 정보가 들어 있어 버전·노드·시점이 맞지 않으면 cluster가 깨질 수 있습니다.

이 부분에서 확인할 것: 성공 기준: cluster green 목표, failed/unassigned shard 0, 필요한 index·alias·정책 복원.

미러 data는 보관본, 복구 원본은 Snapshot

노드별 /application/elk/data 복사는 cluster 일관성을 보장하지 않는다. target에서는 미러 data를 삭제하지 않고 hold로 옮긴 뒤 빈 working data로 새 cluster를 만들고, 완료된 Elasticsearch Snapshot만 restore한다.

11.1 Elasticsearch 7.8 global state 판정

아주 쉽게 설명하면: 11.1 Elasticsearch 7.8 global state 판정

Snapshot 안에 어떤 '전체 설정'을 넣고 뺄지 정하는 단계입니다.

global state는 여러 index에 공통으로 영향을 주는 template-pipeline-persistent 설정 등을 포함할 수 있습니다. 무조건 전체를 복원하면 target 설정을 덮어쓸 수 있고, 무조건 빼면 검색이 달라질 수 있습니다.

따라서 index 데이터와 global state를 분리해 판단하고, template-pipeline-ILM-SLM을 별도로 export/import 하고 비교합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: Elasticsearch node-cluster 정보, index-shard-alias 목록, Snapshot repository, template-pipeline-ILM-SLM 정책입니다. Elasticsearch는 index를 여러 shard로 나누어 저장하므로 data 폴더를 일반 파일처럼 복사하지 않습니다.

실제 흐름: target에서 새 빈 cluster를 준비하고 repository를 등록한 뒤 성공한 Snapshot을 restore합니다. 복원 후 cluster health-node-index-shard를 보고, 정책 객체는 별도로 비교합니다. green은 shard 배치가 정상이라는 뜻이지 업무 검색까지 자동 보장한다는 뜻은 아닙니다.

1. 새 cluster와 repository를 준비합니다.
2. 필요한 snapshot과 index를 복원합니다.
3. template-pipeline-ILM-SLM을 따로 확인합니다.
4. 대표 검색·정렬·aggregation·재기동을 시험합니다.

raw data 폴더를 피하는 이유: 그 안에는 cluster UUID와 shard 상태 같은 내부 정보가 들어 있어 버전·노드·시점이 맞지 않으면 cluster가 깨질 수 있습니다.

이 부분에서 확인할 것: 승인 기준: 어떤 global state를 복원했는지 목록과 담당자 판단 기록.

항목	7.8 snapshot 판정	본 절차
Persistent cluster settings	<code>include_global_state:true</code> 일 때 포함	전체 자동 복원 금지, target용 allowlist만 수동 적용
Transient settings	포함되지 않음	원칙적으로 재적용하지 않고 필요 항목만 재설정
Legacy/composable template, ingest pipeline, ILM policy	global state 포함 대상	별도 JSON export 후 ID별 명시적 import·비교
Component template	7.8 snapshot 포함 범위만 믿지 않음	<code>_component_template</code> 별도 export/import
SLM policy	7.8 global state 포함 목록에 명시되지 않음	<code>_slm/policy</code> 별도 export/import 필수
Snapshot repository 등록	포함되지 않음	target에서 먼저 수동 등록·verify
<code>.kibana*</code> , <code>.security-*</code>	global state가 아니라 실제 index	실제 physical index를 snapshot 목록에 명시하고 필요 시 별도 restore
YAML/keystore/TLS key/Kibana 설정	snapshot 대상 아님	파일 checksum·권한을 별도 이관

보관 snapshot은 `include_global_state:true`로 만들어 복구 선택지를 남기되, OL9.6 target의 운영 restore는 `false`로 고정한다. old node allocation filter, read-only block, recovery throttle 같은 source persistent setting이 target을 오염시키는 것을 막기 위해서다.

11.2 미러 데이터 격리와 새 cluster bootstrap

아주 쉽게 설명하면: 11.2 미러 데이터 격리와 새 cluster bootstrap

기존 검색 data 폴더를 보관만 하고, target에서 새 검색 창고를 시작하는 단계입니다.

bootstrap은 새 cluster를 처음 만드는 작업입니다. 미러 data는 분석·원복용 hold 영역에 두고, target working data는 깨끗한 경로로 시작합니다.

이렇게 하면 target이 오래된 node 정보를 읽어 잘못된 cluster에 들어가는 위험을 줄일 수 있습니다.

이 부분에서 확인할 것: 금지: source의 data directory를 target working directory로 바로 덮어쓰기.

1. target의 ES, Kibana, Scheduler, 애플리케이션 및 ES writer를 모두 중지하고 source의 9300 transport망 접근을 차단한다.
2. 각 노드의 미리 /application/elk/data를 \${HOLD_ROOT}/elasticsearch/<node>-data.pre-rebuild로 이동한다. _state, node ID, cluster UUID, shard와 translog를 신규 cluster에 섞지 않는다.
3. 빈 /application/elk/data와 /logs/elk에 admin:admin 권한을 적용하고 es1/es2/es3, discovery seed, target cluster name, path.repo를 설정한다.
4. 최초 bootstrap에만 cluster.initial_master_nodes를 쓰고 node 3개, 빈 index 목록, source와 다른 새 cluster UUID를 확인한 뒤 설정에서 제거한다.
5. ILM과 SLM을 중지하고 status가 STOPPED인지 확인한다.

```
POST /_ilm/stop
GET /_ilm/status
POST /_slm/stop
GET /_slm/status
```

11.3 Repository·정책 객체 수동 이관

아주 쉽게 설명하면: 11.3 Repository·정책 객체 수동 이관

Snapshot 창고 주소와 자동 관리 규칙을 **target**에도 등록하는 단계입니다.

repository는 snapshot 파일이 있는 장소입니다. ILM은 오래된 index를 정리하는 규칙이고 SLM은 snapshot을 자동으로 찍는 규칙입니다. 데이터만 복원하고 규칙을 잊으면 target이 예기치 않게 삭제·백업 작업을 할 수 있습니다.

target에서는 repository 권한과 read-only 여부를 확인하고, policy·template·pipeline을 하나씩 비교합니다.

이 부분에서 확인할 것: 성공 기준: 정책 ID·내용·실행 주체·다음 실행 여부가 승인된 상태.

1. repository 등록정보는 복원되지 않으므로 target에 수동 등록한다. source와 repository를 공유한다면 target은 readonly:true로 등록하고 _verify한다.
2. source export에서 ingest pipeline, ILM policy, component template, composable template, legacy template 순으로 target에 ID별 PUT한다. API 응답의 통계·수정시각·version wrapper는 PUT body에서 제거한다.
3. node 이름을 기준으로 한 allocation 조건, source 전용 path·주소·read-only block을 제거하고 object 이름·개수·핵심 JSON을 source와 비교한다.
4. SLM은 repository와 restore가 검증될 때까지 import·시작하지 않는다.

```
PUT /_snapshot/ol_migration_repo
{
  "type": "fs",
  "settings": {
    "location": "/migration-backup/elasticsearch",
    "readonly": true
  }
}

POST /_snapshot/ol_migration_repo/_verify
```

```
PUT /_ilm/policy/<ILM_POLICY_ID>
{
  "policy": { "phases": { "<phase>": { "actions": { } } } }
}

# pipeline/template도 source export의 각 ID와 실제 body를 분리해 PUT한다.
```

11.4 Application·Kibana index restore

아주 쉽게 설명하면: 11.4 Application·Kibana index restore

실제 검색 index를 새 Elasticsearch에 읽어오는 단계입니다.

restore는 snapshot에 들어 있는 index를 target cluster에 다시 만드는 작업입니다. application index와 Kibana index를 구분하고, 이미 존재하는 index와 이름이 겹치지 않는지 확인합니다.

복원 후 index count-mapping-alias-대표 검색-aggregation과 Kibana 화면을 확인합니다.

이 부분에서 확인할 것: NO-GO: restore가 끝나기 전에 애플리케이션 writer를 연결하거나 source와 target을 동시에 쓰게 함.

1. restore 전 target에 같은 이름의 열린 index가 0개인지 확인한다. 미러가 만든 index가 남아 있다면 target 식별 승인 후 hold로 되돌아가 working data를 다시 비우고 bootstrap한다.
2. 승인된 application index의 실제 목록을 include_global_state:false, include_aliases:true로 restore한다.
3. Kibana를 중지한 채 manifest의 실제 .kibana_N, .kibana_task_manager_N physical index와 alias를 별도 restore-검증한다.
4. 보안 기능을 사용한다면 .security-*와 ES keystore/TLS/file-realm 파일을 별도 절차로 복원한다. 보안 index가 필요 없으면 자동 포함하지 않는다.

```
POST /_snapshot/ol_migration_repo/<B1_SNAPSHOT>/_restore?wait_for_completion=true
{
  "indices": "<APPLICATION_INDEX_LIST>",
  "ignore_unavailable": false,
  "include_global_state": false,
  "include_aliases": true,
  "partial": false
}
```

11.5 ILM·SLM 안전 검증

아주 쉽게 설명하면: 11.5 ILM·SLM 안전 검증

자동 삭제와 자동 백업이 target에서 안전한지 시험하는 단계입니다.

ILM이 잘못 켜지면 오래된 index를 삭제할 수 있고, SLM이 잘못 켜지면 target에서 snapshot을 찍거나 source 용 repository를 건드릴 수 있습니다.

리허설 중에는 자동 실행을 제한하고, 정책 explain/API와 로그로 실제 대상·조건·권한을 확인합니다.

이 부분에서 확인할 것: 성공 기준: target에서 의도하지 않은 삭제·snapshot·source 접근이 없음.

1. 복원 index는 원래 creation/rollover 시각과 index.lifecycle.name을 보유할 수 있다. ILM 시작 전에 모든 관리 index의 _ilm/explain을 확인해 현재 phase/age/action과 즉시 delete-rollover-allocation 가능성을 승인한다.
2. ILM policy 이름, rollover alias, template 연결과 target tier/node attribute가 유효한지 확인한다. 잘못된 allocation 조건은 시작 전에 수정한다.
3. 복원용 ol_migration_repo는 read-only로 유지한다. SLM에는 target만 write하는 별도 <TARGET_SLM_REPOSITORY>를 등록·verify한다. 동일 저장소를 넘겨받아야 한다면 source의 write 접근을 완전히 제거한 후에만 target에 단독 write 권한을 준다.
4. SLM policy는 source export의 각 policy 객체만 PUT한다. repository를 <TARGET_SLM_REPOSITORY>로 바꾸고 snapshot 이름·index 목록·schedule-retention을 검토한 뒤 시험 snapshot 한 건을 실행한다.
5. SLM retention이 이관용 snapshot을 삭제하지 않도록 migration repository와 runtime SLM repository를 분리한다.
6. 모든 검증과 승인이 끝난 후에만 ILM, SLM 순서로 시작하고 status와 최근 실행 결과를 확인한다.

```
PUT /_slm/policy/<SLM_POLICY_ID>
{
  "schedule": "<APPROVED_CRON>",
  "name": "<APPROVED_SNAPSHOT_NAME_PATTERN>",
  "repository": "<TARGET_SLM_REPOSITORY>",
  "config": { "indices": ["<APPROVED_INDEX_PATTERN>"] },
  "retention": { "expire_after": "<APPROVED_RETENTION>" }
}
```

```
GET /<RESTORED_INDEX_PATTERN>/_ilm/explain
GET /_ilm/policy
GET /_slm/policy
# TARGET_SLM_REPOSITORY는 target 단독 writer여야 한다.
POST /_ilm/start
POST /_slm/start
```

최종 검증 기준: node 3, green, unassigned shard 0, recovery 완료, index 이름·count·mapping·settings·alias 일치, shard 분산, 대표 검색·정렬·aggregation 및 CRUD, ILM explain 승인, SLM 시험 snapshot 성공, ES 한 노드 재기동 후 green 복귀. Kibana는 별도 index/alias 검증 뒤 101 한 대에서 기동하고 saved object를 확인한다.

12. 애플리케이션·Scheduler 기동

아주 쉽게 설명하면: 12. 애플리케이션·Scheduler 기동

서비스들을 서로 의존하는 순서대로 켜는 단계입니다.

Master·Engine·Gateway·CMS는 혼자서 완성되는 프로그램이 아닙니다. DB·Redis·검색 시스템을 먼저 준비하고, 애플리케이션이 올바른 주소를 보도록 한 뒤 화면과 업무 흐름을 확인합니다.

Scheduler는 다른 서비스와 DB를 자동으로 바꿀 수 있으므로 마지막에 한 대만 제한적으로 켵니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: Scheduler 실행 node, 실행 주기, job 이름, 대상 DB·Solr·Redis·Elasticsearch, 진행 중 job 목록입니다. Scheduler는 사람이 버튼을 누르지 않아도 시간에 맞춰 일하는 자동 직원입니다.

실제 흐름: source와 target의 자동 직원이 동시에 출근하지 않게 합니다. 진행 중 job이 끝났는지 확인하고 source Scheduler를 중지합니다. 리허설에서는 승인된 한 대만 제한 기동하고 배포·ES 로그 삭제 job은 별도 승인합니다.

1. 실제 실행 node를 찾습니다.
2. 진행 중·예약 job을 확인합니다.
3. source Scheduler를 멈춥니다.
4. target 한 대에서 필요한 job만 시험합니다.
5. DB·검색·cache 변화를 비교합니다.

큰 위험: 101·102·103이 동시에 실행되면 같은 통계나 배포가 중복될 수 있습니다. 그래서 Scheduler는 전체 검증의 마지막에 한 대만 켵니다.

이 부분에서 확인할 것: 성공 기준: 각 서비스 health와 로그 정상, 중복 Scheduler 없음.

12.1 첫 기동 전

아주 쉽게 설명하면: 12.1 첫 기동 전

전원을 켜기 전에 마지막으로 확인하는 단계입니다.

target hostname·IP·설정파일·환경변수·실행 사용자·자동기동 여부를 확인합니다. source 주소가 설정에 남아 있으면 target 서비스가 원본 DB나 검색 cluster에 접속할 수 있습니다.

기동 전에 서비스별 의존 대상과 포트를 적어 두고, 실패하면 status와 journal을 저장합니다.

조금 더 자세하: 실제 작업의 흐름

들어오는 자료: DB·Redis·검색 서비스 health 결과, JAR, 환경변수, 설정파일, 실행 사용자, 서비스 의존관계입니다. 애플리케이션은 다른 서비스에 전화를 거는 프로그램입니다.

실제 흐름: 데이터 서비스가 준비된 뒤 Master·Engine·Gateway·CMS·Chat UI를 연결합니다. 각 서비스마다 프로세스·port·API·로그를 확인하고 source 주소를 보고 있지 않은지 검색합니다.

1. 파일과 설정 주소를 확인합니다.
2. 의존 서비스가 정상인지 봅니다.
3. 한 서비스씩 기동합니다.
4. API와 업무 흐름을 검증합니다.

기억할 점: systemctl active는 1차 신호일 뿐입니다. 실제 성공은 API·연결·로그인·검색·시뮬레이터까지 확인해야 합니다.

이 부분에서 확인할 것: 멈춤 기준: source endpoint가 남아 있거나 target 격리가 확인되지 않음.

- ☐ 모든 설정의 DB/Redis/ZK/Solr/ES URL이 격리된 target을 가리키며 source IP나 운영 VIP가 남지 않았다.
- ☐ 복원된 DB의 tbl_manage_schema.prefix_url, schema_name, user_name과 tbl_manage_tenant.manage_schema_id를 전수 확인했다. tenant 접속지는 YAML이 아니라 이 테이블에서 동적으로 조립되므로 source 주소가 하나라도 남으면 기동 금지한다.
- ☐ ibot.cms.domain, master.api.url, Engine URL과 target VM의 outbound ACL을 확인해 원본 DB·서비스로의 egress를 차단했다.
- ☐ JAR 파일명과 checksum이 미리 직전 VM manifest와 일치한다.
- ☐ profile은 tc, core-tc이며 로그는 각 /logs/<module> 아래로 기록된다.
- ☐ master/CMS 첫 검증 기동은 schema 변경을 막기 위해 ddl-auto=none 또는 validate override가 적용됐다.
- ☐ OS와 JVM의 timezone이 모두 Asia/Seoul인지 확인했다. Quartz cron은 timezone을 별도 지정하지 않고 코드가 LocalDateTime/LocalDateTime.now()를 사용한다.
- ☐ L4/VIP에 target이 아직 등록되지 않았고 테스트 호출이 외부 시스템으로 나가지 않는다.

12.2 기동 원칙

아주 쉽게 설명하면: 12.2 기동 원칙

한 번에 다 켜지 않고 한 단계씩 확인하는 규칙입니다.

기반 서비스가 정상인지 확인하고 다음 서비스를 켭니다. 그래야 오류가 DB 때문인지, Redis 때문인지, 앱 설정 때문인지 찾기 쉽습니다.

start-restart-enable은 모두 상태를 바꾸는 명령이므로 승인된 작업창에서만 사용합니다. 'active' 표시만 보지 말고 포트-API·로그·실제 업무까지 확인합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: DB-Redis-검색 서비스 health 결과, JAR, 환경변수, 설정파일, 실행 사용자, 서비스 의존관계입니다. 애플리케이션은 다른 서비스에 전화를 거는 프로그램입니다.

실제 흐름: 데이터 서비스가 준비된 뒤 Master·Engine·Gateway·CMS·Chat UI를 연결합니다. 각 서비스마다 프로세스·port-API·로그를 확인하고 source 주소를 보고 있지 않은지 검색합니다.

1. 파일과 설정 주소를 확인합니다.
2. 의존 서비스가 정상인지 봅니다.
3. 한 서비스씩 기동합니다.
4. API와 업무 흐름을 검증합니다.

기억할 점: `systemctl active`는 1차 신호일 뿐입니다. 실제 성공은 API·연결·로그인·검색·시뮬레이터까지 확인해야 합니다.

이 부분에서 확인할 것: 원칙: 한 번에 하나의 변경, 변경 전후 상태 기록.

데이터 서비스가 모두 합격한 후 master → engine → gateway → CMS → chat-ui 순으로 한 단계씩 기동한다. 각 단계에서 port, health URL, DB connection, 로그 error를 확인한 뒤 다음 단계로 간다.

Scheduler는 가장 마지막에 101 한 대만 기동한다. 소스 분석상 Quartz 설정에 JDBC JobStore 기반 클러스터/락이나 동시 실행 방지 보장이 없어서 3대 동시 기동 시 같은 job이 중복 실행될 수 있다. 102/103의 Scheduler는 정지·비활성 상태로 유지한다.

Scheduler 고위험 job

RESERVE_DEPLOY_JOB은 매분 예약 배포를 찾아 CMS를 호출하고 DB·Solr active/standby·Redis cache·Engine 알림까지 바꿀 수 있다. ES_LOG_BACKUP_JOB은 ES snapshot 생성 후 보존기간이 지난 index를 삭제할 수 있다. target endpoint 격리와 승인 없이 Scheduler 전체를 켜지 않는다.

12.3 Scheduler 제한 시험

아주 쉽게 설명하면: 12.3 Scheduler 제한 시험

자동 작업을 아주 작은 범위에서 시험하는 단계입니다.

Scheduler는 정해진 시간마다 통계 생성, 배포, 로그 백업 등을 할 수 있습니다. 101·102·103이 모두 켜지면 같은 작업이 중복 실행될 수 있습니다.

리허설에서는 승인된 한 VM의 Scheduler만 켜고, 위험한 배포·삭제 job은 비활성으로 둡니다. 실행 전후 DB·Solr·ES·Redis와 로그를 비교합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: Scheduler 실행 node, 실행 주기, job 이름, 대상 DB·Solr·Redis·Elasticsearch, 진행 중 job 목록입니다. Scheduler는 사람이 버튼을 누르지 않아도 시간에 맞춰 일하는 자동 직원입니다.

실제 흐름: source와 target의 자동 직원이 동시에 출근하지 않게 합니다. 진행 중 job이 끝났는지 확인하고 source Scheduler를 중지합니다. 리허설에서는 승인된 한 대만 제한 기동하고 배포·ES 로그 삭제 job은 별도 승인합니다.

1. 실제 실행 node를 찾습니다.
2. 진행 중·예약 job을 확인합니다.
3. source Scheduler를 멈춥니다.
4. target 한 대에서 필요한 job만 시험합니다.
5. DB·검색·cache 변화를 비교합니다.

큰 위험: 101·102·103이 동시에 실행되면 같은 통계나 배포가 중복될 수 있습니다. 그래서 Scheduler는 전체 검증의 마지막에 한 대만 켵니다.

이 부분에서 확인할 것: 성공 기준: 한 대만 실행, 진행 중 job 없음, 예상 범위의 데이터만 변경.

1. `spring.quartz.auto-startup=false`를 적용한 별도 시험 설정으로 자동 cron을 막고 101 한 대에서만 프로세스를 올린다. `spring.batch.job.enabled=false`만으로 Quartz cron이 멈춘다고 가정하지 않는다.
2. 읽기·파생 통계 job 한 개를 격리 데이터로 수동 실행한다. 예약 배포와 ES log backup은 리허설 승인 전 실행 금지한다.
3. 실행 전후 DB/ES/Solr/Redis count와 `tbl_batch_log` 한 건의 시작·종료·상태를 기록한다.
4. 수동 `/executeScheduler`는 job별 먹등성과 코드 매핑이 확인된 범위에만 쓴다. 현재 seed에는 소스에 없는 AnswerLink 계열 클래스도 있어 `ClassNotFoundException` 가능성이 있다.
5. `learn recommend`는 수동 from/to를 읽지 않고 중복 insert 가능성이 있다. `dialog/satisfaction` 통계도 bulk insert/unique 충돌 위험이 있어 자동 보정하지 않는다. Knowledge 통계의 여러 날짜 보정은 날짜 증가 로직 위험 때문에 반드시 1일 단위로 실행·대조한다.
6. 시험이 끝나면 Scheduler를 다시 끄고 B1 완료 때까지 정지 상태를 유지한다.

ES_LOG_BACKUP_JOB은 실패 상태를 만들더라도 종료부에서 성공으로 덮을 수 있는 코드 경로가 확인됐다. 따라서 tbl_batch_log의 성공 표시만 믿지 말고 Elasticsearch _snapshot API의 실제 snapshot 상태와 ES 로그를 직접 확인한다.

13. T0 — Oracle Linux 9.6 리허설 시험

아주 쉽게 설명하면: 13. T0 — Oracle Linux 9.6 리허설 시험

T0는 본 공연 전에 하는 전체 리허설입니다.

T0는 화면 하나가 뜨는지 보는 시험이 아니라, 로그인·문서 처리·검색·챗봇·배포·시뮬레이터를 target에서 끝까지 실행해 보는 과정입니다.

실패하더라도 운영 사용자는 source를 계속 사용하고, target만 고칩니다. 테스트 케이스와 기대 결과를 미리 정해야 '성공했다'고 말할 수 있습니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: 테스트 IP, 테스트 계정, 입력 데이터, 기대 결과, 로그 위치, PASS/FAIL 기준입니다. 기준이 없으면 화면 하나 뜬 것을 성공으로 착각합니다.

실제 흐름: VM-OS → port → 제품 health → 애플리케이션 API → 로그인·검색·배포·챗봇 → 시뮬레이터 순서로 시험합니다. 실패하면 다음 단계로 넘어가지 않고 그때의 로그와 설정을 저장합니다.

1. 프로세스가 실행 중인지 확인합니다.
2. port가 열렸는지 확인합니다.
3. API와 cluster를 확인합니다.
4. 업무 시나리오를 실행합니다.
5. 장애 후 복구도 시험합니다.

PASS의 뜻: 명령어가 끝났다는 뜻이 아니라 예상한 데이터·화면·로그·응답이 나온다는 뜻입니다.

이 부분에서 확인할 것: PASS 조건: 데이터·기능·재기동·장애복구·시뮬레이터 기준 모두 통과.

분류	시험	합격 기준	증적
OS/Java	JDK8, timezone, DNS, 권한, SELinux enforcing, ulimit, 재부팅 후 자동기동	오류/거부 없음, 지정 계 정·경로 유지	명령 결과, audit/log
Native binary	Redis/MariaDB/ES launcher 의존성, plugin/JNI/native library	missing library/symbol/SSL 오 류 없음	version, ldd, journal
MariaDB	미러 MyISAM check, A그룹 선택 재적 재, B그룹 후보 PK 교정, 제외 table 불 변, 한글/emoji·AUTO_INCREMENT	손상 0, A/B 검증 일치, 비대상 변경 0	INSERT/rollback SQL·count/checksum
Redis	미러 cluster 재결합 또는 빈 3M/3R 재 구성, 시스템 cache 재배포, replica 재 기동	16384 slot·cluster_state ok·배 포 cache 조회 정상	cluster info/nodes·배포 결과
ZK/Solr	leader/follower, collection/alias/count, ktnori, CRUD, 단일 노드 재기동	모든 replica active, 한 글 검색 및 복구 정상	API JSON, 로그
ES/Kibana	빈 cluster+snapshot restore, count/mapping/alias, ILM explain, SLM 시험, CRUD, node restart, dashboard	unassigned 0, 정책·결 과 일치, green 복귀	API JSON, policy diff, 화면
Application	각 health/로그, DB-Redis-Solr-ES 연계, cms/chat-ui UI, 주요 사용자 시나리오	source endpoint 접근 0, 오류율 기준 이하	호출 결과, 접근 로그
Scheduler	101 한 대 제한 기동, 승인된 job 1회, 중복·미종료 확인	예상 row만 생성, 종료 상태 정상	before/after SQL
L4	격리 health pool, 한 앱 인스턴스 중지 시 다른 인스턴스 라우팅	정상 제외/복귀, 세션 정 책 충족	L4 상태·호출 로그
복구	MariaDB A그룹 rollback/B그룹 PK 원 복, Redis cache 재배포, Solr collection·ES snapshot restore 재현	DB 전체 폐기 없이 선택 원복, 필수 collection/index 복구	소요시간, 전/후 hash

리허설 완료의 정의

화면이 뜨는 것만으로 완료하지 않는다. 빈 클러스터 생성부터 backup restore, 데이터 비교, 단일 노드 장애-재합류, 전체 절차 재현과 예상 소요시간 측정까지 통과해야 한다.

14. B1 — 최종 데이터 동기화

아주 쉽게 설명하면: 14. B1 — 최종 데이터 동기화

운영 전환 직전에 마지막 변경분을 맞추는 단계입니다.

B0 리허설 이후에도 source에서 새 문서·통계·배치 기록이 생길 수 있습니다. B1은 신규 요청과 writer를 멈춘 뒤 마지막 순간의 데이터를 고정하는 백업입니다.

B1이 끝난 뒤 target을 다시 검증하고, 그 다음에만 운영 트래픽을 target으로 보냅니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: 원본의 현재 상태, 백업 저장 위치, 생성 시각, checksum, 복원 방법입니다. 백업은 파일 한 개를 복사하는 행동이 아니라 실패했을 때 돌아갈 지점을 만드는 일입니다.

B0와 B1: B0는 리허설 전 복원 연습용이고, B1은 최종 전환 직전 writer와 Scheduler를 멈춘 뒤 마지막 변경분을 고정하는 용도입니다. 이름이 비슷해도 시점과 목적이 다릅니다.

1. 백업 대상을 정합니다.
2. 파일·시각·크기를 기록합니다.
3. SHA-256으로 변조 여부를 봅니다.
4. target에서 실제 복원합니다.
5. 성공·실패와 승인을 남깁니다.

초보자 원칙: '백업 성공' 메시지만 믿지 말고 목록·checksum·복원 결과까지 확인합니다.

이 부분에서 확인할 것: 멈춤 기준: writer 정지와 진행 중 job 0건이 확인되지 않으면 B1을 만들지 않음.

MariaDB는 미러 DB 전체를 유지하고 A그룹 data-only INSERT SQL과 B그룹 영향 PK만 동기화한다. Redis 데이터는 동기화하지 않고 정상 cluster 위에 cache를 재배포한다. Solr는 모든 collection의 B1 BACKUP/RESTORE, Elasticsearch는 B1 Snapshot/Restore와 정책 객체의 명시적 이관을 수행한다.

14.1 최종 쓰기 중지

아주 쉽게 설명하면: 14.1 최종 쓰기 중지

source에서 데이터가 더 변하지 않도록 잠시 멈추는 단계입니다.

사용자 요청과 Scheduler가 계속 들어오면 source와 target의 데이터가 서로 달라집니다. 그래서 점검창 동안 신규 요청을 막고, 진행 중 작업이 끝났는지 확인합니다.

서비스를 막는 순서와 읽기 전용 전환 여부는 운영 담당자가 결정합니다. 강제 종료보다 정상 완료·writer 차단을 우선합니다.

이 부분에서 확인할 것: 성공 기준: Scheduler·writer 정지, 진행 중 job 없음, 중단 시각 기록.

1. L4에서 신규 사용자 쓰기를 차단하고 업무 점검창을 시작한다. T_FINAL을 기록한다.
2. **source Scheduler를 가장 먼저 중지**한다. 진행 중 job이 끝났는지 애플리케이션 로그와 `tbl_batch_log.batch_enddt IS NULL` 조회로 확인한다.
3. source gateway/CMS/master/engine/chat-ui 등 모든 writer를 중지하거나 read-only로 만든다.
4. 예약 배포 대기·진행 상태, 학습/배포 작업, ES snapshot 작업이 진행 중이 아닌지 업무 테이블과 로그에서 확인한다.
5. 예약 배포 조회 결과가 있으면 Scheduler 재기동 직후 실행될 수 있으므로 목록을 업무 담당자에게 승인받고, 미승인 건이 있으면 계속 정지한다.
6. DB/Solr/ES에서 2회 연속 count/position을 확인해 쓰기가 멈췄음을 기록한다. Redis는 cache 기준 건수만 채증하며 최종 동기화 기준으로 쓰지 않는다.

14.2 B1 최종 선택 백업

아주 쉽게 설명하면: 14.2 B1 최종 선택 백업

마지막 백업 묶음을 만드는 단계입니다.

MariaDB는 승인된 선택 범위의 INSERT SQL, Solr는 모든 collection BACKUP, Elasticsearch는 index Snapshot과 정책 export를 만듭니다. 제품마다 안전한 백업 방법이 다릅니다.

파일명·생성 시각·checksum·완료 상태를 기록합니다. 백업 하나라도 실패하면 운영 전환하지 않습니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: 원본의 현재 상태, 백업 저장 위치, 생성 시각, checksum, 복원 방법입니다. 백업은 파일 한 개를 복사하는 행동이 아니라 실패했을 때 돌아갈 지점을 만드는 일입니다.

B0와 B1: B0는 리허설 전 복원 연습용이고, B1은 최종 전환 직전 writer와 Scheduler를 멈춘 뒤 마지막 변경분을 고정하는 용도입니다. 이름이 비슷해도 시점과 목적이 다릅니다.

1. 백업 대상을 정합니다.
2. 파일·시각·크기를 기록합니다.
3. SHA-256으로 변조 여부를 봅니다.
4. target에서 실제 복원합니다.
5. 성공·실패와 승인을 남깁니다.

초보자 원칙: '백업 성공' 메시지만 믿지 말고 목록·checksum·복원 결과까지 확인합니다.

이 부분에서 확인할 것: NO-GO: 일부 collection/index만 백업되었거나 snapshot failed shard가 있음.

1. MariaDB A그룹만 data-only-complete INSERT 형식으로 추출하고 B그룹의 source/target 후보 PK 합집합, source 후보행 SQL, count/checksum/AUTO_INCREMENT manifest를 만든다. 전체 DB dump는 하지 않는다.
2. Redis는 B1 RDB/AOF를 이관하지 않는다. 현재 cluster topology와 key count는 참고 증적으로만 남긴다.
3. Solr의 source collection 이름을 다시 전수 조회하고 모든 collection에 hard commit 후 새 이름으로 BACKUP한다. configset/alias/count와 비동기 completed 결과를 collection별로 남긴다.
4. Elasticsearch의 ILM/SLM을 중지하고 기존 status를 기록한다. 정책·template·pipeline·settings를 다시 export한 뒤 include_global_state:true인 final snapshot을 만들고 SUCCESS/failed shard 0/index 목록을 확인한다.
5. 노드별 CMS upload 등 파일성 데이터를 다시 archive/checksum한다.

모든 B1 산출물의 종료 시각과 checksum이 확정될 때까지 source를 변경·재기동하거나 L4를 target으로 넘기지 않는다. 전환이 취소되면 source의 ILM/SLM을 기존 status로 되돌린다.

14.3 MariaDB R1 선택 재적재

아주 쉽게 설명하면: 14.3 MariaDB R1 선택 재적재

최종 고정된 source 데이터를 target DB에 적용하는 단계입니다.

R1은 B1을 target에 반영한 결과입니다. A그룹은 승인된 전량 교체, B그룹은 후보 PK 선택 교정, 그 외 업무 원본은 보존합니다.

적용 후 source와 target의 count-checksum-기본키-AUTO_INCREMENT-논리 orphan을 비교합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: source DB의 현재 데이터, target DB의 table 구조, Scheduler가 건드리는 table 목록, 그리고 승인된 적용 범위입니다. DB는 단순한 파일 창고가 아니라 여러 프로그램이 계속 읽고 쓰는 업무 장부이므로, '파일이 복사됐나'보다 '어떤 행이 같은가'를 확인합니다.

실제 흐름: 먼저 읽기 명령으로 접속-table-행 수-진행 중 작업을 확인합니다. 그다음 writer와 Scheduler를 멈추고 source에서 승인된 data-only INSERT SQL을 만듭니다. target에서는 승인된 범위만 preclear하고 INSERT 한 뒤 source와 count-기본키-checksum을 비교합니다. 나머지 업무 원본 table은 건드리지 않는 것이 기본입니다.

1. **읽기:** DB가 살아 있고 구조가 맞는지 봅니다.
2. **고정:** 더 이상 행이 변하지 않도록 writer를 멈춥니다.
3. **적용:** A그룹 전량 또는 B그룹 후보 PK만 처리합니다.
4. **비교:** 다르면 성공으로 보지 않고 멈춥니다.

가장 중요한 이유: MyISAM에서는 실패하면 DB가 자동으로 원래 상태로 돌아온다고 가정하면 안 됩니다. 따라서 지우기 전에 before-image와 원복 INSERT SQL을 먼저 준비합니다.

이 부분에서 확인할 것: 성공 기준: DB 담당자 승인 전에는 애플리케이션 writer를 켜지 않음.

1. target Scheduler와 모든 application writer를 중지하되 미리 MariaDB는 격리 상태로 유지한다.
2. A그룹 target 현 데이터를 data-only rollback SQL로 추출하고 B1 SQL을 staging DB에 선적재해 검증한다.
3. A그룹 table의 데이터만 DELETE FROM으로 비운 뒤 검증된 B1 INSERT SQL을 적재한다. DB/schema/table을 drop-truncate하지 않는다.
4. B그룹은 source/target 후보 PK 합집합의 target before-image를 저장한 뒤 tbl_manage_deploy, tbl_learn_recommend, tbl_deploy 후보행만 source 최종값으로 교체한다.
5. tbl_bot은 영향 bot의 active/standby deploy-learn 컬럼만 source staging 값으로 UPDATE한다.
6. A/B 이외 table 불변, A그룹 count/checksum/AUTO_INCREMENT, B그룹 후보행과 논리 orphan을 확인한다. MyISAM 적용 실패 시 target before-image로 역순 원복한다.

14.4 Redis·Solr·Elasticsearch R1

아주 쉽게 설명하면: 14.4 Redis·Solr·Elasticsearch R1

세 검색·cache 계층의 최종 상태를 target에서 맞추는 단계입니다.

Redis는 재결합 또는 cache 재생성, Solr는 모든 collection restore, Elasticsearch는 B1 snapshot restore와 정책 검증을 수행합니다.

서비스가 켜져 있는지만 보지 말고 cluster 상태·문서 수·대표 검색·alias·TTL·shard를 확인합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: Redis 노드의 주소·port·master/replica 역할·node ID·slot 상태와 AOF/RDB 파일 정보입니다. Redis가 cache만 저장하는지, 로그인 세션·queue·업무 상태도 저장하는지는 애플리케이션 설정으로 확인합니다.

실제 흐름: PONG은 한 노드가 대답했다는 뜻이고, cluster_state:ok와 모든 slot 정상은 집단 전체가 일할 준비가 됐다는 뜻입니다. 기존 cluster 재결합이 안전하면 유지하고, 주소·node ID·slot이 꼬이면 빈 3 master/3 replica를 구성한 뒤 cache를 다시 만듭니다.

1. 노드와 역할을 기록합니다.
2. source와 target이 동시에 같은 cluster가 되지 않게 합니다.
3. 재결합 또는 새 cluster 중 하나를 선택합니다.
4. cache 재생성 뒤 TTL·대표 기능을 확인합니다.

주의: 'Redis니까 지워도 된다'가 아닙니다. cache라면 다시 만들 수 있지만 세션·queue라면 초기화가 곧 업무 영향일 수 있으므로 담당자 승인이 필요합니다.

이 부분에서 확인할 것: NO-GO: 원본 endpoint 접근, collection/index 누락, Redis slot 미완료.

1. DB 선택 재적재 보고서가 승인된 뒤 target cluster writer와 관련 application을 계속 정지한다.
2. Redis/Solr/ZK/ES 리허설 상태는 \${HOLD_ROOT}/rehearsal-<RUN_ID>로 안전 보관한다.
3. Redis는 B1 데이터를 import하지 않는다. cluster 합격값을 다시 확인하고, 리허설 cache가 남아 있으면 시스템 배포 기능의 full replace 여부에 따라 target cache를 승인된 방식으로 비운 뒤 전체 재배포한다.
4. ZooKeeper/Solr를 빈 ensemble/cloud로 다시 만들고 B1 configset과 모든 collection을 restore한다. 하나라도 누락되면 중단한다.
5. Elasticsearch의 미리·리허설 data를 working 경로에서 제외하고 빈 신규 cluster를 만든다. repository를 다시 등록하고 B1 application/Kibana index를 include_global_state:false로 restore한 뒤 정책 객체를 명시적으로 적용한다.
6. 파일성 데이터는 B1 manifest와 DB 참조를 기준으로 승인된 최종 세트를 배치한다.
7. B1 manifest와 target을 비교한 뒤 application을 기동한다. Scheduler는 여전히 OFF다.

14.5 최종 데이터 검증

아주 쉽게 설명하면: 14.5 최종 데이터 검증

운영 전환 전에 마지막 비교표를 만드는 단계입니다.

데이터가 같고 기능도 같아야 합니다. DB 행 수만 같아도 검색 index가 빠졌을 수 있고, 화면이 떠도 일부 업무 API가 실패할 수 있습니다.

데이터 검증과 업무 시나리오 검증을 모두 통과한 뒤에야 L4/VIP/DNS를 바꿉니다.

이 부분에서 확인할 것: 최종 산출물: 비교 결과, 로그, checksum, 담당자 서명, GO/NO-GO 판단.

- ☐ master DB와 모든 tenant DB에서 A그룹 source/target 일치, B그룹 후보 PK 일치, 비대상 table 불변, AUTO_INCREMENT, 논리 orphan 0, 미완료 batch/deploy 상태 확인.
- ☐ tbl_batch_log와 9개 통계 table은 source B1로 재적재되었고 target 리허설 이력은 rollback SQL에만 보존됨.
- ☐ tbl_manage_deploy, tbl_deploy, tbl_bot, tbl_learn_recommend은 후보 PK/상태 컬럼만 교정되었고 DB-Solr-Redis-Engine 상태와 함께 업무 승인됨.
- ☐ Redis 16384 slot·3M/3R·대표 cache/TTL, Solr collection 이름·count·alias·ktnori, ES index count/mapping/alias·ILM explain·SLM 시험 결과.
- ☐ 모든 application이 source IP에 연결하지 않고 target hostname/VIP 설정만 사용.

binlog는 후보 발견용일 뿐

ROW binlog/GTID가 사용 가능하면 B0_SAFE 이후 B그룹 후보 PK 누락 여부를 교차 확인할 수 있다. binlog 전체 replay나 DB 전체 동기화에는 사용하지 않으며, 최종 적용 범위는 승인된 A그룹과 B그룹 후보 manifest로 제한한다.

15. 권장 중지·기동 순서

아주 쉽게 설명하면: 15. 권장 중지·기동 순서

서비스를 끄고 켜는 순서를 적은 시간표입니다.

중지할 때는 새 요청과 자동 작업을 먼저 막고, 기동할 때는 DB·조정 서비스·검색·애플리케이션 순서로 아래에서 위로 올라갑니다. 이는 집의 수도·전기부터 켜고 가전제품을 켜는 것과 비슷합니다.

실제 서비스 관계가 다르면 담당자의 운영 매뉴얼을 우선하지만, Scheduler를 마지막 한 대만 켜는 원칙은 지킵니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: DB-Redis-검색 서비스 health 결과, JAR, 환경변수, 설정파일, 실행 사용자, 서비스 의존관계입니다. 애플리케이션은 다른 서비스에 전화를 거는 프로그램입니다.

실제 흐름: 데이터 서비스가 준비된 뒤 Master·Engine·Gateway·CMS·Chat UI를 연결합니다. 각 서비스마다 프로세스·port·API·로그를 확인하고 source 주소를 보고 있지 않은지 검색합니다.

1. 파일과 설정 주소를 확인합니다.
2. 의존 서비스가 정상인지 봅니다.
3. 한 서비스씩 기동합니다.
4. API와 업무 흐름을 검증합니다.

기억할 점: `systemctl active`는 1차 신호일 뿐입니다. 실제 성공은 API·연결·로그인·검색·시뮬레이터까지 확인해야 합니다.

이 부분에서 확인할 것: 확인할 것: 각 단계 사이의 health, 포트, 로그, source 접근 차단.

최종 중지

1. L4 신규 요청 차단
2. Scheduler
3. chat-ui / CMS / gateway
4. engine / master
5. Kibana
6. MariaDB A/B 선택 data-only SQL
7. Solr 모든 collection B1 BACKUP
8. Elasticsearch 정책 export-B1 Snapshot
9. Elasticsearch → Solr → ZooKeeper
10. Redis topology 채증 후 중지
11. MariaDB는 강제 종료하지 않고 writer 차단 상태 유지

복원 후 기동

1. MariaDB
2. ZooKeeper → Solr
3. Elasticsearch → Kibana
4. Redis cluster
5. master
6. engine
7. gateway
8. CMS
9. chat-ui
10. 시스템 배포 기능으로 Redis cache 재배포
11. L4 내부 health 검증
12. Scheduler 101 한 대, 최종 승인 후

관찰

1. 내부 smoke test
2. L4 target pool 소량 투입
3. 오류-latency-data write 확인
4. 전체 전환
5. 집중 관찰
6. source는 write 차단 상태 보존

Redis cluster, Solr collection restore, ES snapshot restore는 일부 병렬화할 수 있으나 최종 cache 재배포는 DB·Solr·ES와 관련 애플리케이션 검증 후 수행한다. 위 순서는 안전한 기본값이다.

16. GO/NO-GO와 롤백

아주 쉽게 설명하면: 16. GO/NO-GO와 롤백

GO는 다음 단계로 가고, **NO-GO**는 멈추는 신호입니다.

이관에서는 '일단 해 보고 문제면 고치자'보다, 미리 정한 통과 기준이 중요합니다. 백업·격리·데이터·기능·전환 단계마다 통과 기준을 둡니다.

롤백은 target을 지우는 일이 아니라, target 트래픽을 차단하고 source로 돌아가는 절차입니다. target에서 이미 쓰기가 발생했다면 데이터가 두 방향으로 달라질 수 있으므로 먼저 write 상태를 판단합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: 통과 기준, 중단 기준, 책임자, source 보존 상태, target의 write 발생 여부입니다. GO는 다음 단계 진행 승인이고 NO-GO는 해결될 때까지 멈추는 신호입니다.

실제 롤백: 오류를 기록하고 신규 요청을 막은 뒤 target Scheduler와 writer를 중지합니다. target write가 없으면 L4·DNS·VIP를 source로 되돌립니다. write가 있었다면 데이터 충돌을 먼저 판단합니다.

1. 오류 기준과 로그를 기록합니다.
2. target 요청을 막습니다.
3. target write를 멈춥니다.
4. source 경로를 복구합니다.
5. 사용자 업무를 재검증합니다.

중요: 전환 후 target에 쓰기가 있었다면 주소만 되돌리는 것으로는 최신 입력을 보존할 수 없습니다.

이 부분에서 확인할 것: 원칙: source와 백업은 안정화가 끝날 때까지 보존.

게이트	GO 조건	NO-GO 시 조치
G0 채증/백업	DB 선택 INSERT SQL, Redis topology, Solr 모든 collection backup, ES snapshot·정책 export 성공	미러 착수/target 작업 중지
G1 격리	source-target 동일 IP·cluster·VIP 통신 불가	target 서비스 기동 금지
G2 기준 상태	MariaDB 미러 무결성, Redis 파일 세트 판정, ZK/Solr/ES 미러 data hold 완료	raw hold에서 working copy 재생성 후 원인 분석
G3 B0 복원	DB A/B 선택 적용 검증, Redis 16384 slot, Solr collection 전부 active/count 일치, ES index·정책 검증	DB는 rollback SQL 원복, Redis cache 재생성, Solr/ES는 backup으로 재복원
G4 T0 시험	데이터·기능·재기동·장애 시험 합격	원인 보완 후 리허설 반복
G5 B1	writer 정지, DB 선택 SQL·Solr 전체 collection·ES snapshot/policy checksum과 성공 상태 확정	L4 전환 금지, source 유지·정책 상태 원복
G6 R1	DB A/B 일치·비대상 불변, Redis cache 재배포, Solr/ES B1 일치, source endpoint 접근 0	DB 선택 원복 또는 해당 cluster 재복원
G7 cutover	health/오류율/응답/핵심 write 정상	아래 rollback 판단

16.1 롤백 기준

아주 쉽게 설명하면: 16.1 롤백 기준

문제가 생겼을 때 돌아가는 방법을 설명하는 부분입니다.

L4 전환 전에는 사용자가 source를 계속 쓰므로 target을 중단하고 고치면 됩니다. L4 전환 후 target에 쓰기가 없으면 경로를 source로 돌리기 비교적 쉽지만, target에 쓰기가 있었다면 단순히 주소만 되돌리면 데이터가 사라질 수 있습니다.

그래서 롤백 시점과 write 발생 여부를 기록하고, DB-Redis-Solr-Elasticsearch별로 어떤 데이터를 보존하거나 폐기할지 담당자 승인을 받습니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: 통과 기준, 중단 기준, 책임자, source 보존 상태, target의 write 발생 여부입니다. GO는 다음 단계 진행 승인이고 NO-GO는 해결될 때까지 멈추는 신호입니다.

실제 롤백: 오류를 기록하고 신규 요청을 막은 뒤 target Scheduler와 writer를 중지합니다. target write가 없으면 L4-DNS-VIP를 source로 되돌립니다. write가 있었다면 데이터 충돌을 먼저 판단합니다.

1. 오류 기준과 로그를 기록합니다.
2. target 요청을 막습니다.
3. target write를 멈춥니다.
4. source 경로를 복구합니다.
5. 사용자 업무를 재검증합니다.

중요: 전환 후 target에 쓰기가 있었다면 주소만 되돌리는 것으로는 최신 입력을 보존할 수 없습니다.

이 부분에서 확인할 것: 금지: target write 후 아무 검토 없이 DNS/L4만 source로 되돌리기.

L4 전환 전

target application/Scheduler를 중지하고 실패한 cluster 상태를 보관한다. DB A그룹 적용 실패는 target-preclean.rollback.insert.sql로 전량 원복하고, B그룹은 후보 PK before-image를 역순 적용한 뒤 count/checksum을 확인한다. MyISAM에는 transaction rollback이 없으므로 준비된 원복 SQL이 필수다. source에는 cluster reset이나 데이터 변경 명령을 실행하지 않는다.

L4 전환 후, target write 발생 전

target writer를 중지하고 L4/VIP를 source로 되돌린 뒤 source writer를 허용한다. target DB 자체를 폐기하지 않고 전환 시각, 무쓰기 확인, 마지막 선택 재적재 manifest와 rollback SQL을 보존한다. source ILM/SLM을 중지했다면 기존 상태로 되돌린다.

L4 전환 후, target write 발생 후

단순 무손실 롤백 불가

source와 target writer를 모두 중지한다. target write를 폐기할 수 있으면 손실 시간대를 승인·기록하고 source로 되돌린다. 보존해야 하면 역동기화 설계 없이 라우팅만 되돌리지 않는다. 현재 구성만으로 Redis-Solr-Elasticsearch의 범용 역방향 CDC는 없다.

최종 전환 승인

B1 checksum 확인: _____

R1 데이터 비교: _____

애플리케이션 승인: _____

DB/미들웨어 승인: _____

L4 전환 책임자·시각: _____

롤백 판단 한계시각: _____

부록 A. Scheduler·배치 데이터 분류

아주 쉽게 설명하면: 부록 A. Scheduler·배치 데이터 분류

어떤 DB 데이터가 자동 작업의 영향을 받는지 정리한 참고표입니다.

모든 데이터가 같은 성격은 아닙니다. 업무 원본은 보존해야 하고, 통계·배치 기록은 다시 만들 수 있으며, 배포 상태는 여러 시스템과 함께 움직일 수 있습니다.

분류표는 데이터 삭제 허가서가 아닙니다. 실제 배포 JAR·DB schema·로그에서 쓰기 동작을 확인하고, 담당자 승인으로 최종 범위를 확정합니다.

조금 더 자세하: 실제 작업의 흐름

들어오는 자료: Scheduler 실행 node, 실행 주기, job 이름, 대상 DB·Solr·Redis·Elasticsearch, 진행 중 job 목록입니다. Scheduler는 사람이 버튼을 누르지 않아도 시간에 맞춰 일하는 자동 직원입니다.

실제 흐름: source와 target의 자동 직원이 동시에 출근하지 않게 합니다. 진행 중 job이 끝났는지 확인하고 source Scheduler를 중지합니다. 리허설에서는 승인된 한 대만 제한 기동하고 배포·ES 로그 삭제 job은 별도 승인합니다.

1. 실제 실행 node를 찾습니다.
2. 진행 중·예약 job을 확인합니다.
3. source Scheduler를 멈춥니다.
4. target 한 대에서 필요한 job만 시험합니다.
5. DB·검색·cache 변화를 비교합니다.

큰 위험: 101·102·103이 동시에 실행되면 같은 통계나 배포가 중복될 수 있습니다. 그래서 Scheduler는 전체 검증의 마지막에 한 대만 컵니다.

이 부분에서 확인할 것: 성공 기준: 보존·선택 교정·전량 교체·고위험 job의 경계가 명확함.

로컬 소스 /Users/seoriguri/IdeaProjects/meritz의 Git HEAD 1502c3c72eec와 현재 schema를 기준으로 분류했다. 이름은 실제 DB의 소문자 표기를 사용했다. 이 소스가 VM의 배포 JAR과 같은 revision인지 checksum/build 기록으로 대조하고, 다르면 배포본 기준으로 재분석한다.

분류	대상	최종 동기화 원칙
업무 원본-보존	tbl_manage_tenant, tbl_manage_schema, tbl_manage_bot, tbl_code_group, tbl_code_cd, tbl_chat_channel, tbl_intent, tbl_sentence, tbl_chat_node, tbl_api, tbl_learning_info 및 만족도 원천 테이블	미러 데이터를 그대로 유지한다. Scheduler 선택 재적재 범위에 포함하지 않으며 전후 count/checksum 불변을 확인한다.
공용 상태-선택 교정 B	tbl_learn_recommend, tbl_deploy, tbl_manage_deploy, tbl_bot	Scheduler와 CMS/Master가 함께 쓰므로 전체를 비우지 않는다. B0_SAFE 이후 source/target 후보 PK 합집합만 source 최종값으로 교정하며, tbl_bot은 active/standby deploy-learn 상태 컬럼만 UPDATE한다.
Scheduler 전용-전량 교체 A	각 tenant의 tbl_user_statistics, tbl_slot_statistics, tbl_dialog_statistics, tbl_api_statistics, tbl_scenario_statistics, tbl_scenario_node_statistics, tbl_knowledge_statistics, tbl_satisfaction_statistics, tbl_top10_dialog_statistics	target table의 데이터만 DELETE하고 source B1 data-only INSERT SQL을 전량 재적재한다. 이전 통계는 ES 원천 로그 보존기간 때문에 임의 재산출하지 않는다.
Scheduler 전용-전량 교체 A	tbl_batch_log	master DB의 target 데이터를 비우고 source B1 INSERT SQL로 전량 재적재한다. target 시험 이력은 적용 전 rollback SQL에만 보존하며 최종 DB에는 남기지 않는다.
운영 파생	tbl_manage_monitoring_response	저장 코드는 있으나 현재 MonitoringJob.executeInternal() 본문은 주석 처리된 것으로 분석됨. 기본 선택 대상에서 제외하며 배포 JAR에서 실제 write가 확인된 경우에만 추가한다.
Batch 메타	표준 BATCH_*, tbl_job_instance	현재 DB에는 표준 BATCH_*가 관찰되지 않았고 tbl_job_instance 활성 사

분류	대상	최종 동기화 원칙
		용도 확인되지 않았다. 기본 대상에서 제외하고 실제 배포 JAR과 DB에서 활성 쓰기가 확인되면 별도 변경 승인으로 정확한 table만 추가한다.
고위험 동작	RESERVE_DEPLOY_JOB, ES_LOG_BACKUP_JOB	예약 배포는 DB·Solr·Redis·Engine을 함께 변경하며 ES 로그 백업은 index를 삭제할 수 있다. 리허설 기본 비활성, 최종 target 검증 후 승인 기동.

A.1 Job별 재실행 위험

아주 쉽게 설명하면: A.1 Job별 재실행 위험

자동 작업을 다시 실행해도 되는지 판단하는 표입니다.

어떤 job은 같은 시간 범위를 다시 계산해도 괜찮을 수 있지만, 어떤 job은 같은 행을 두 번 넣거나 배포를 두 번 실행할 수 있습니다. 이름이 비슷해도 위험이 다릅니다.

재실행 전에 중복키·upsert 여부·from/to 범위·DB와 검색의 동시 변경 여부를 확인합니다. 위험한 배포와 로그 삭제 job은 기본적으로 비활성으로 둡니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: Scheduler 실행 node, 실행 주기, job 이름, 대상 DB·Solr·Redis·Elasticsearch, 진행 중 job 목록입니다. Scheduler는 사람이 버튼을 누르지 않아도 시간에 맞춰 일하는 자동 직원입니다.

실제 흐름: source와 target의 자동 직원이 동시에 출근하지 않게 합니다. 진행 중 job이 끝났는지 확인하고 source Scheduler를 중지합니다. 리허설에서는 승인된 한 대만 제한 기동하고 배포·ES 로그 삭제 job은 별도 승인합니다.

1. 실제 실행 node를 찾습니다.
2. 진행 중·예약 job을 확인합니다.
3. source Scheduler를 멈춥니다.
4. target 한 대에서 필요한 job만 시험합니다.
5. DB·검색·cache 변화를 비교합니다.

큰 위험: 101·102·103이 동시에 실행되면 같은 통계나 배포가 중복될 수 있습니다. 그래서 Scheduler는 전체 검증의 마지막에 한 대만 컵니다.

이 부분에서 확인할 것: NO-GO: 멍등성·중복 실행 안전성이 확인되지 않은 job 재실행.

Job	기본 범위	주요 쓰기	판정
USER / SLOT / API / SCENARIO / KNOWLEDGE 통계	직전 1 시간 또는 전일	각 통계표 upsert 성격	단일 Scheduler에서 비교적 재산출 가능하나 동시 실행 race 금지
DIALOG / SATISFACTION 통계	직전 1 시간 또는 전일	일부 기존 조회 없는 bulk insert	같은 범위 재실행 시 unique 충돌 가능. 범위를 백업·삭제한 뒤 한 번만 실행
TOP10_DIALOG	직전 1 시간	환경 조건에 따라 insert/upsert	현재 TC 조건에서 재실행 위험. 결과 대조 필수
LEARN_RECOMMEND	직전 1 시간	tbl_1learn_recommend append	unique 없음, from/to 미지원. 과거 누락분 자동 catch-up 대상 아님
RESERVE_DEPLOY	매분	DB·Solr·Redis·Engine 복합 변경	업무 승인 전 비활성. 양쪽 동시 실행 절대 금지
ES_LOG_BACKUP	매일	snapshot 및 오래된 index 삭제	파괴적. batch log 성공값이 실제 성공을 보장하지 않으므로 snapshot API로 판정

A.2 Scheduler 컷오버 체크

아주 쉽게 설명하면: A.2 Scheduler 컷오버 체크

운영 전환 직전 Scheduler가 한 대만 실행되는지 확인하는 목록입니다.

batch log에 끝나지 않은 작업이 없는지, 예약 배포가 대기 중인지, target Scheduler가 꺼져 있는지 확인합니다. 이 목록을 통과하지 못하면 target이 운영 데이터를 몰래 바꿀 수 있습니다.

마지막 SQL 조회는 진행 중 예약 배포 후보를 찾는 예입니다. 조회 결과가 있다고 바로 삭제하거나 실행하지 말고 담당자가 상태를 판단합니다.

조금 더 자세히: 실제 작업의 흐름

들어오는 자료: Scheduler 실행 node, 실행 주기, job 이름, 대상 DB·Solr·Redis·Elasticsearch, 진행 중 job 목록입니다. Scheduler는 사람이 버튼을 누르지 않아도 시간에 맞춰 일하는 자동 직원입니다.

실제 흐름: source와 target의 자동 직원이 동시에 출근하지 않게 합니다. 진행 중 job이 끝났는지 확인하고 source Scheduler를 중지합니다. 리허설에서는 승인된 한 대만 제한 기동하고 배포·ES 로그 삭제 job은 별도 승인합니다.

1. 실제 실행 node를 찾습니다.
2. 진행 중·예약 job을 확인합니다.
3. source Scheduler를 멈춥니다.
4. target 한 대에서 필요한 job만 시험합니다.
5. DB·검색·cache 변화를 비교합니다.

큰 위험: 101·102·103이 동시에 실행되면 같은 통계나 배포가 중복될 수 있습니다. 그래서 Scheduler는 전체 검증의 마지막에 한 대만 컷니다.

이 부분에서 확인할 것: 성공 기준: source stop 기록, 진행 job 0, target 101 한 대만 승인 기동.

- ☐ source Scheduler stop 시각과 마지막 로그를 기록했다.
- ☐ tbl_batch_log에서 batch_enddt IS NULL 또는 진행 상태 row가 0이다.
- ☐ tbl_manage_deploy/tbl_deploy에 진행 중 예약 배포가 없다.
- ☐ B1 선택 재적재 후 A그룹은 source count/checksum과 일치하고 자연키 중복·미종료 batch가 없다.
- ☐ B그룹은 후보 PK만 교정됐고 그 밖의 업무행과 비대상 table은 변경되지 않았다.
- ☐ 미리 중 중단 구간을 자동 catch-up하지 않았다. 필요한 job·기간·명등성을 업무 담당자가 승인했다.
- ☐ target Scheduler는 101 한 대만 기동되고 102/103은 정지 상태다.

```
SELECT id, bot_id, deploy_time
FROM tbl_deploy
WHERE COALESCE(is_del, 'N') = 'N'
      AND result_message IS NULL
      AND deploy_status = 'STANDBY'
      AND deploy_type = 'SCHEDULE'
      AND deploy_time <= NOW();
```

일반 통계 job은 현재 시각 기준 직전 시간/전일 범위를 계산하며 tbl_batch_log를 checkpoint로 읽지 않는 것으로 분석됐다. 따라서 중단 시간만큼 자동으로 정확히 따라잡는다고 가정하면 안 된다.

부록 B. 증적·인수 체크리스트

아주 쉽게 설명하면: 부록 B. 증적·인수 체크리스트

작업이 끝났다는 증거를 모으는 목록입니다.

‘정상적으로 됐다’고 말하려면 어떤 서버에서 어떤 명령을 실행했고, 어떤 backup-checksum·테스트 결과가 나왔는지 남아 있어야 합니다.

나중에 장애가 생기거나 다음 사람이 인수할 때 이 자료가 작업의 기억이 됩니다. 비밀번호·토큰은 증적에 남기지 않고 가립니다.

이 부분에서 확인할 것: 완료 기준: VM·OS·파일·DB·Redis·Solr·ES·업무·L4·롤백 증적 모두 보관.

산출물	권장 파일명	확인
작업 변수·대상 식별	00-run-sheet-<RUN_ID>.txt	☐
노드/OS/Java/port/service 현황	01-host-manifest-<node>.txt	☐
JAR/config/upload checksum	02-files-SHA256SUMS	☐
MariaDB A그룹 B0/B1 data-only INSERT SQL	mariadb/<DB>.{B0 B1}.{batch-log statistics}.insert.sql	☐
MariaDB B그룹 후보·target 원복·적용 manifest	mariadb/{candidate-pk,target-preclean,applied,rollback}.*	☐
Redis 미리 topology·상태파일/RDB/AOF checksum·cache 배포 결과	redis/{mirror-manifest,cluster-result,cache-deploy-result}.*	☐
Solr 모든 collection B0/B1-configset·alias/count	solr/{B0 B1}/...	☐
ES B0/B1 snapshot·index 및 ILM/SLM/template/pipeline diff	elasticsearch/{B0 B1}-manifest.json, elasticsearch/policies/	☐
T0/R1 시험 결과	test/{T0 R1}-result.html	☐
L4 cutover/rollback 증적	network/cutover-<RUN_ID>.txt	☐
오류·예외·의사결정 기록	decision-log-<RUN_ID>.md	☐

부록 C. 참고 문서

아주 쉽게 설명하면: 부록 C. 참고 문서

각 제품의 공식 설명서로 다시 확인하는 곳입니다.

이관 중 명령어의 정확한 옵션과 제품 버전별 동작은 공식 문서가 기준입니다. 런북은 현재 환경에 맞춘 작업 초안이고, 제품이 업데이트되거나 환경이 다르면 공식 문서를 함께 확인해야 합니다.

특히 DB dump, Redis persistence, Solr backup, ZooKeeper quorum, Elasticsearch snapshot은 잘못 사용하면 데이터 손실로 이어질 수 있으므로 버전이 맞는 문서를 읽습니다.

이 부분에서 확인할 것: 확인할 것: 제품 버전, 명령어 버전, 사내 보안·운영 표준과의 일치.

- Oracle Linux 9.6 Release Notes — target OS 기준 릴리스 정보와 알려진 이슈 확인.
- Rocky Linux 8.4 Release Notes — Rocky Linux의 첫 릴리스가 8.4임을 확인하는 공식 자료.
- MariaDB: mariadb-dump — MyISAM에는 single transaction이 일관성을 보장하지 않으며 --lock-all-tables는 dump 동안 global read lock을 사용한다.
- Redis: Persistence — AOF/RDB 우선순위와 복구 특성.
- Redis Cluster specification — node ID, slot, cluster bus와 gossip 동작.
- Apache Solr: Backup and Restore — SolrCloud BACKUP/RESTORE와 shared filesystem, hard commit.
- Apache ZooKeeper 3.6.1 Administrator's Guide — myid, 신규 ensemble initialize marker, quorum 구성.
- Elasticsearch 7.8: Create Snapshot API — global state에 persistent setting-template-pipeline-ILM이 포함되고 transient setting-repository는 제외됨.
- Elasticsearch 7.8: Restore Snapshot API — 선택 index와 include_global_state:false 복원.
- Elasticsearch 7.8: Get SLM Policy API — SLM policy 별도 export용.
- Elasticsearch 7.8: Register Repository — target repository 재등록과 다중 cluster 공유 시 read-only 원칙.
- Elastic: Filesystem backup warning — node data directory 복사는 일관된 cluster backup이 아님.

완료 판정 문장

“EL7 계열 또는 Rocky Linux 8로 추정된 AS-IS 배포판을 실측하고 Oracle Linux 9.6 target으로 전환했다. 미리 MariaDB를 유지한 채 Scheduler 전용 A그룹만 source B1 INSERT SQL로 재적재하고 공용 B그룹은 영향 PK만 교정했다. Redis는 별도 데이터 이관 없이 정상 cluster에 cache를 재배포했으며, Solr 모든 collection과 Elasticsearch B1 snapshot-ILM/SLM 정책을 복원·검증했다. 주요 업무 시나리오와 단일 노드 장애복구를 통한 뒤 Scheduler는 101 한 대에서만 최종 승인 후 기동했다.”

본 문서는 현재 테스트베드와 소스 코드 분석을 바탕으로 만든 실행 초안이다. 실제 운영 환경의 주소·용량·테이블 engine·제품

패치·보안 정책을 작업 직전 재확인하고, 모든 파괴적 단계는 대상 식별과 책임자 승인을 거쳐야 한다.